

World Action Planner: Generalizable Decision-Making with Action-Conditioned World Models

Xiangcheng Zhang
Harvard University
xiangchengzhang@fas.harvard.edu

Yilun Du
Harvard University
ydu@seas.harvard.edu

Abstract

Building generalizable agents for diverse applications remains a fundamental challenge. While imitation learning-based policies succeed in specific training environments, they often fail to generalize to novel scenes and tasks. In this work, we propose **World Action Planner**, a robot planning system that leverages the reasoning capabilities of Vision-Language Models (VLMs) and the physical grounding of a multi-task pose-image conditioned world model. Our system enables an agent to propose initial action plans and iteratively refine them via optimization and search, reasoning over imagined world model rollouts. We demonstrate that our approach achieves superior performance across compositional tasks, new layouts, and zero-shot generalization scenarios, significantly outperforming state-of-the-art end-to-end policy models such as VLAs and WAMs. Project website at worldactionplanner.github.io

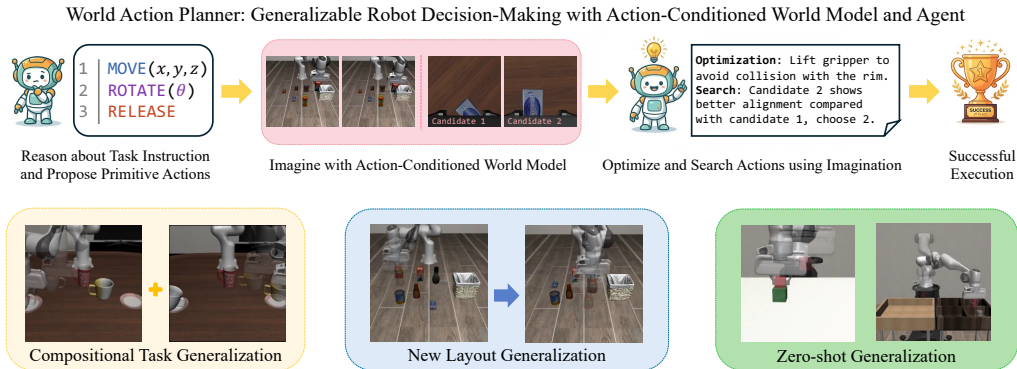


Figure 1: We introduce **World Action Planner**, a planning system in which the agent reasons about and refines its action plans via world model imagination, generalizing to novel scenarios and solving new tasks at test time.

1 Introduction

Developing generalizable robotic agents capable of mastering diverse tasks in heterogeneous environments remains a fundamental challenge in AI. Recent progress has been driven by end-to-end (E2E) imitation learning-based policies that leverage pretrained Vision-Language Models (VLMs) [27] or video generation models [78] as backbones. However, these approaches are often fundamentally limited by the scope of their demonstration data during training. For instance, a policy trained only on individual pick-and-place atomic trajectories often fails to navigate between objects when executing composed, long-horizon tasks. Furthermore, E2E models frequently overfit spurious motion patterns in demonstrations; if training data consistently depicts grasping at a specific coordinate, the policy may continue targeting that position even after the object has moved.

Consequently, these models struggle in novel scenarios that demand systematic reasoning or flexible planning beyond the specific trajectory distributions seen during training.

In this work, we depart from the E2E imitation learning paradigm. Instead, we propose a principled planning system centered on a world model that enables an agent to propose, simulate, and iteratively refine action plans. Our approach is rooted in classical robotics, where environment abstractions and robot actions are represented through programs [47] to allow for modular composition and generalization. While modern foundation models can generate high-level plans using such abstractions, they often lack an understanding of physical causality and dynamics, thus failing to directly generate executable actions. By integrating an action-conditioned world model, we enable systematic model-based planning that bridges the gap between high-level reasoning and physical execution. We show theoretically that model-based planning offers superior multi-task generalization compared to imitation learning. By training a generalizable world model on diverse trajectories, the system can synthesize new action sequences for unseen tasks and configurations via imagination-based planning, which remains challenging for end-to-end (E2E) policies tethered to expert demonstration trajectories.

In this work, we propose **World Action Planner** (Fig. 1), a robot planning system that orchestrates VLM agents and action-conditioned world models to solve new tasks with new scenes and layouts. In this framework, the VLM agent proposes initial action sequences which are then optimized through iterative interaction with the world model. We propose a systematic action planning pipeline that integrates agent-driven feedback and corrections based on world model imagined rollouts, and searches for plausible actions by imagining future trajectories of different candidates. To facilitate this, we develop a generalizable, *pose-image conditioned* multi-view world model capable of precise control over robot actions and high-fidelity simulation of physical interactions.

We demonstrate the generalization capability of our method across compositional tasks, new layouts, and zero-shot scenarios in simulation environments. In compositional long-horizon tasks, our agent successfully manages transitions between sub-tasks by proposing and refining maneuvers through imagination, whereas end-to-end models such as VLAs often stagnate after the first sub-task. In spatial generalization with modified object positions, our method accurately identifies the new target object position and completes the task, while imitation learning policies tend to reach for training-set object coordinates. By leveraging imaginations from the action-conditioned world model, our system detects physical risks such as collisions, and optimizes proposed actions for safe execution. Furthermore, we demonstrate that local search identifies feasible actions and states for fine-grained manipulation, such as mug grasping and cube stacking, by reasoning over imagined outcomes of different candidates.

We summarize our main contributions as follows:

- (i) We introduce a generalizable, *pose-image conditioned* robot world model and demonstrate its superior performance when generalizing to new actions, scenes, and across diverse robots.
- (ii) We provide theoretical analysis showing that model-based planning outperforms imitation learning in multi-task generalization for both tabular settings and linear function approximation.
- (iii) We propose **World Action Planner**, a planning system that orchestrates VLM agents and world models for systematic action proposal, optimization, and search via world model imagination. Our results demonstrate that the proposed method significantly outperforms state-of-the-art policies across compositional tasks, new layouts, and zero-shot test-time generalization scenarios.

2 Related Works

Here, we review the most closely related works. For an extended discussion, see Appendix A.

Robot World Models. Recent robot world models leverage pre-trained diffusion-based video backbones [69, 76] and condition on low-dimensional actions via cross-attention [14] or AdaLN-Zero modulation [86, 57], while other approaches utilize textual descriptions [75, 2, 3]. [66] demonstrated that pose-conditioned world models can effectively simulate OOD and unsafe scenarios, thereby serving as evaluators for robot policies. [70] also adopted robot pose-image conditioning; however, while they extracted ground-truth poses from future frames, we compute them through dynamics to enable planning via imagination. Finally, concurrent work [31] conditions on rendered robot arm images, while our pose skeleton image representation is more computationally efficient and robust.

Planning with VLMs and World Models. Foundation VLMs are typically integrated into robotics either via high level planning [44, 26, 25] or as backbones for Vision-Language-Action

(VLA) models [27, 36]. To enhance robot policies, [29, 56, 31, 19] propose sampling-based or gradient-based action optimization using the imagination from the world model. More recently, world-action models (WAM) [35, 78, 40] fine-tune video generative models using demonstration data to generate future actions as well as future frames, and video planners [67, 11, 9, 15] leverage synthetic task completion videos from video generative models and extract the actions using inverse-dynamics models. Our approach is distinguished by the use of a VLM agent for planning and reasoning, leveraging imagined rollouts from an *action-conditioned* world model to optimize and verify the actions in new environments and tasks.

3 Methods

Here we describe our world model in Section 3.1 and the world action planner in Section 3.2.

3.1 Pose-Image Conditioning for Action-Controlled Robot World Modeling

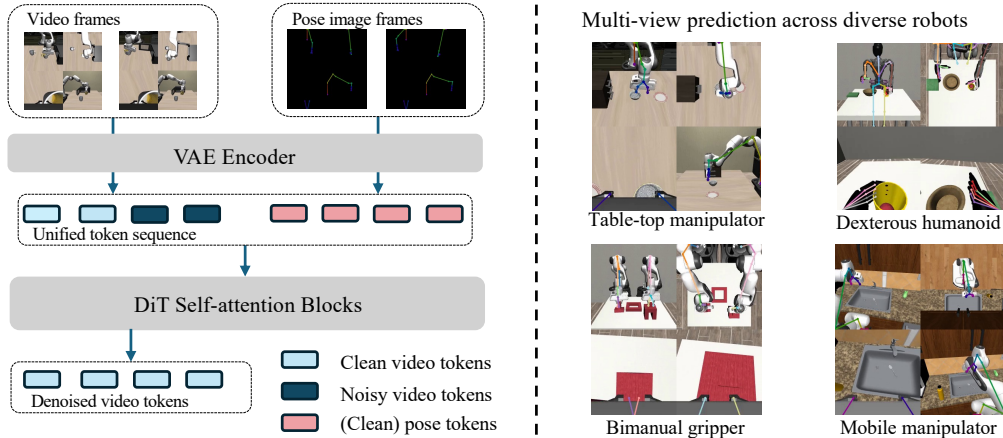


Figure 2: **Illustration of our pose-image conditioned world model**

Prior robot action-conditioned video diffusion models typically condition on low-dimensional action vectors through AdaLN-Zero modulation [57, 86] or cross-attention [14]. However, such conditioning often fails to generalize to novel actions. In our approach, we pre-compute future robot joint positions via forward dynamics of the actions and render these positions as a pose skeleton image from the corresponding camera viewpoint. More details about pose-image conditioning are in Appendix C.1.1. Inspired by [65], we encode the pose image frames using the VAE of the video model and concatenate the resulting pose image tokens with the video tokens into a unified sequence. We adopt multi-view prediction by concatenating third-person and wrist-view camera feeds into a grid for each video and pose-image frame [14], so that the model can infer the relative 3D position of each robot joint from multi-view images. We train the model using diffusion-forcing [63, 9] with the flow-matching objective [42], where we add independent random noise to the history video tokens and uniform noise to the future video tokens, while keeping the pose-image tokens noise-free for conditioning.

3.2 World Action Planner

To achieve generalizable robotic decision-making, we develop an action planning system that orchestrates VLM agents, action-conditioned world models, and manipulation policies, instead of end-to-end imitation learning. The pseudocode is in Alg. 1, with more details in Appendix C.2.

Agent Action Proposal. First, we leverage the extensive world knowledge within foundation VLMs to propose action primitives. Specifically, similar to prior works [45, 44], the VLM generates a sequence of primitive actions, such as MOVE, ROTATE, GRASP, and RELEASE. For the MOVE actions, the VLM is asked to identify the target gripper position across *multi-view* images [7], and then we triangulate these 2D pixel coordinates into 3D space, eliminating the requirement for explicit depth information [44, 45]. We employ a low-level policy as the robot controller, which processes the current and VLM-proposed target end-effector poses as inputs to generate action chunks for execution.

Algorithm 1 World Action Planner (WAP)

Require: VLM Agent, low-dimensional robot controller ϕ , world-model WM , (Optional) policy π ,
Input: Task description ℓ , initial state s , environment env ,
while not done **do**
 $\triangleright$ **Agent Action Proposal**
 Propose primitive actions using the VLM agent, and then translate into robot actions using
 controller: $g = \text{Agent}.\text{ProposeActions}(s, \ell)$, $a = \phi(g, s_{\text{proprio}})$
 $\triangleright$ **Global Optimization Guided by Agent Feedback**
 Imagine the actions using the world model: $\hat{s}_{\text{next}} = \text{WM}(s, a)$
 Optimize based on agent feedback: $\Delta g = \text{Agent}.\text{Optimize}(\hat{s}_{\text{next}}, \ell)$, $a = \phi(g + \Delta g, s_{\text{proprio}})$
 $\triangleright$ **Local Search with Agent Ranking**
 Generate action candidates: $a_1, a_2, \dots, a_N = \text{GridSearch}(a)$
 Imagine action candidates: $\hat{s}_i = \text{WM}(s, a_i)$, $i = 1, 2, \dots, N$
 if use policy **then**
 Imagine the policy rollout after the candidate action: $\hat{s}_i = \text{WM}(\hat{s}_i, \pi(\hat{s}_i))$, $i = 1, 2, \dots, N$
 end if
 Select the best one using VLM agent: $i^* = \text{argmax Agent.Rank}(\hat{s}_1, \hat{s}_2, \dots, \hat{s}_N, \ell)$
 $\triangleright$ **Execute the Actions**
 Execute the actions after search: $s, \text{done} = \text{env}.\text{Step}(s, a_{i^*})$
 if use policy **then**
 Roll out the policy following the actions a_{i^*} : $s, \text{done} = \text{env}.\text{Step}(s, \pi(s))$
 end if
end while

Global Optimization Guided by Agent Feedback. When proposing primitive actions, VLMs often fail to account for the physical consequences of the resulting trajectory executed. For instance, a naive MOVE command between objects may lead to collisions. To address this, we imagine the action trajectory using our action-conditioned world model and prompt the VLM to evaluate whether the actions are safe and aligned with the intended goal based on the imagined rollout video. If a trajectory is deemed suboptimal, the VLM provides high-level semantic feedback to refine the action sequence [80]. For example, if the gripper risks a collision, the VLM may suggest increasing the height for clearance; similarly, if an object is dropped behind the box, the model might suggest a forward adjustment. The action sequence is then updated through the controller based on this feedback.

Local Search with Agent Ranking. While VLMs provide effective corrective feedback for significant trajectory errors, they often struggle with subtle positioning inaccuracies that impede fine-grained tasks, such as grasping a mug by the rim. Although VLMs exhibit robust semantic understanding of spatial relationships, they inherently lack absolute metric grounding, such as directly outputting precise physical coordinates or distances from visual inputs. To overcome this, we transition to a discriminative selection process: we employ a grid search to sample a set of candidate actions and leverage the VLM’s evaluative capabilities to identify the optimal trajectory, using the imagined rollout videos from the action-conditioned world model. For fine-grained manipulation, we further roll out a diffusion policy [10] to complete the grasp once the gripper is proximal to the target; we also imagine this process within the world model to identify the optimal state for subsequent policy execution.

Policies as Tools. In our World Action Planner framework, we treat existing imitation learning policies as modular *tools*, enabling the integration of diverse models, such as diffusion policies [10], VLAs [27], or WAMs [78]. This approach allows us to directly leverage powerful generative policies for in-distribution tasks where demonstrations are available and synthesize novel actions using our full model-based planning system in OOD scenarios. This paradigm shift of viewing policies as tools provides a robust pathway for expanding the generalization frontier of robot decision-making.

4 Theoretical Insights

In this section, we provide theoretical justifications that model-based planning can facilitate efficient multi-task generalization in scenarios where imitation learning may fail, focusing on the tabular setting (Section 4.1) and with linear function approximation (Section 4.2). Proof can be found in Appendix B.

Setup. We model multi-task learning as a contextual MDP $\mathcal{M} = \{\mathcal{M}_c, c \in \mathcal{C}\}$ [18], where $c \in \mathcal{C}$ represents the context (task) provided to the agent. Each task is an MDP $\mathcal{M}_c = \{\mathcal{S}, \mathcal{A}, \mathbb{P}, r_c, \rho, H\}$, where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, respectively. Here, $\mathbb{P}$ denotes the universal dynamics shared across tasks, while r_c is the task-specific reward function. We consider episodic MDPs with horizon H and initial state distribution ρ . In our setting, the agent first interacts with the environment for K episodes. At test time, given a random context c , the agent produces a policy $\hat{\pi}(\cdot; c)$ based on the collected data. We then evaluate the performance using the suboptimality gap $V_{\mathcal{M}_c}^* - V_{\mathcal{M}_c}^{\hat{\pi}(\cdot; c)}$, where the value function $V_{\mathcal{M}}^\pi = \mathbb{E}_{\mathcal{M}, \pi, \rho} \left[\sum_{h=1}^H r(s_h, a_h) \right]$ is the expected cumulative reward.

During the data collection phase, the imitation learning agent can query an expert policy for any task c to roll out in each episode, ultimately outputting a policy $\pi \in \Pi$ that minimizes the empirical error $\Pr(\pi(s; c) \neq a)$ over all data points $(s, a; c)$ in the dataset. In contrast, the model-based planning agent interacts with the environment in each episode k using a context c_k and policy π_k of its choosing. This agent then outputs estimated models $\hat{\mathbb{P}}$ and $\hat{r}$. At test time, given a random context c , the agent derives the optimal policy within the estimated MDP $\hat{\mathcal{M}}_c = (\mathcal{S}, \mathcal{A}, \hat{\mathbb{P}}, \hat{r}_c, \rho, H)$ through planning.

4.1 Tabular Setting

Prior work [64] has established that for certain classes of MDPs, model-based methods can achieve exponential improvements in sample complexity over model-free approaches. However, these results typically rely on strong structural assumptions regarding the model class; in the standard tabular setting, model-free and model-based methods are often shown to be information-theoretically equivalent. In contrast, we demonstrate that in the tabular setting with known rewards for each task, imitation learning can suffer from constant suboptimality if the number of tasks scales linearly with the data budget. Conversely, model-based exploration can efficiently learn the universal dynamics and produce near-optimal policies through planning given new tasks.

Theorem 1. *When the reward r_c is known for any $c \in \mathcal{C}$, (i) there exists a model-based algorithm such that, given K episodes of interaction, it outputs a policy with $\tilde{O}\left(\frac{1}{\sqrt{K}}\right)$ suboptimality gap for any task c , and (ii) there exists a contextual MDP instance such that the average suboptimality gap of the imitation learning agent across tasks $c \in \mathcal{C}$ is at least $\Omega\left(\frac{|\mathcal{C}|}{K}\right)$.*

We note that in real-world applications, a VLM can be utilized to evaluate the terminal state of an episode to provide the reward for each task. In the single-task setting, imitation learning may achieve a superior $1/K$ suboptimality gap compared to the $1/\sqrt{K}$ gap of a model-based planning agent, owing to the expert policy. However, because the expert policy is distinct across different tasks, the imitation learning approach suffers from a suboptimality gap that scales linearly with the total number of tasks.

4.2 Linear Function Approximation

We now consider the linear MDP setting as defined in [34], which utilizes a feature map $\phi(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^d$. Under this framework, the random (noisy) reward and transitions are represented as linear functions: $\mathbb{P}(\cdot | s, a) = \langle \phi(s, a), \boldsymbol{\mu}(\cdot) \rangle$ and $\mathbb{E}[r(s, a)] = \langle \phi(s, a), \theta \rangle$, where $\boldsymbol{\mu}$ denotes unknown measures over $\mathcal{S}$ and θ is an unknown vector. The formal details are provided in Definition 1. To facilitate learning rewards across tasks, we assume a known task representation map $\psi(c) : \mathcal{C} \rightarrow \mathbb{R}^{d'}$ exists, such that there is a matrix $\Theta \in \mathbb{R}^{d \times d'}$ satisfying $\mathbb{E}[r(s, a; c)] = \phi(s, a)^\top \Theta \psi(c)$ for all tasks.

These feature-based assumptions are common in practice; for instance, $\phi(s, a)$ may represent visual features [53] used to model world dynamics [85, 5], while $\psi(c)$ represents language features derived from task instructions c . In such cases, the reward is often approximated by the alignment between language and visual features [58]. To ensure that the feature space could be explored at every layer h , we assume $\kappa := \min_h \sup_\pi \lambda_{\min}(\mathbb{E}_\pi[\phi(s_h, a_h)\phi(s_h, a_h)^\top]) > 0$, which is standard as in [39, 37]. Given the above assumptions, we have the following result for the model-based planning agent:

Theorem 2. *There exists a model-based algorithm that learns the dynamics and multi-task rewards, and outputs a policy with $\tilde{O}\left(\frac{1}{\kappa\sqrt{K}}\right)$ suboptimality gap for any task $c \in \mathcal{C}$ with K episodes of data.*

Next, we analyze the imitation learning agent. We consider a policy class consisting of multiclass classifier functions $\pi(f)(s; c) = \operatorname{argmax}_a f(s, a; c)$ associated with a function $f \in \mathcal{F}$ [59]. While the policy is optimal if $f = Q^*$ is the optimal Q-function, we do not strictly require $Q^* \in \mathcal{F}$ to ensure the optimal policy is realizable in $\mathcal{F}$. Under function approximation, we assume each $f \in \mathcal{F}$ is defined using the feature maps: $f(s, a; c) = f(\phi(s, a), \psi(c))$. We obtain the following results:

Theorem 3. *In the single-task setting, the optimal policy is a linear policy [49, 59], where the function $f^*(s, a)$ with $\pi^*(s) = \operatorname{argmax}_a f^*(s, a)$ can be represented as a linear function of $\phi(s, a)$, and the imitation learning agent’s output policy achieves $\tilde{O}(\frac{1}{K})$ suboptimality. In the multi-task setting, however, there exists an instance where the optimal policy $\pi^*(s; c) = \operatorname{argmax}_a f(\phi(s, a), \psi(c))$ cannot be realized by any function f that is an n -degree polynomial in the feature $\psi(c)$ with $n \leq |\mathcal{A}|$.*

These findings indicate that while the optimal policy possesses a simple linear structure in the single-task setting that allows for efficient learning, this structure breaks down in the multi-task setting, where the dependency on task features becomes highly complex. This aligns with empirical observations: training a generalist expert policy is often difficult due to distinct behaviors across tasks, whereas a generalizable multi-task reward model can be effectively obtained using pretrained vision and language models, and we can obtain optimal policies with model-based planning across tasks.

5 Experiments

Here, we provide comprehensive experiments of both action-conditioned world modeling with our robot pose-image conditioning in Section 5.1, and generalizable robot planning with our World Action Planner in Section 5.2. Experiment details and qualitative examples are in Appendix D.

5.1 Action-Conditioned World Modeling

We evaluate our pose-image conditioned world model by addressing three primary questions: **Q1:** Can we accurately generate multi-view future images conditioned on robot pose-images? **Q2:** Does pose-image conditioning improve generalization compared with low-dimensional action conditioning approaches? **Q3:** Can the model learn dynamics across diverse robot embodiments and action spaces?

Setup. We validate our approach across four simulation suites: LIBERO [43], Robocasa [51], MimicGen [50], and DexMimicGen [32]. To generalize to exploratory actions, we perturb demonstration trajectories with Gaussian noise following the MPPI algorithm [73], a standard for world-model-based control [19, 29]. Each task uses 100 training trajectories (including perturbed versions) and 10 held-out evaluation trajectories. We employ Wan-T2V-1.3B [69] as the backbone, finetuning for 10K steps with a global batch size of 64. Models process 21 history frames¹ at 7 FPS to predict 20 future frames at 20 FPS. Input consists of four camera views concatenated into a 2×2 grid (224px per view). For inference efficiency, we use 20 diffusion steps for sampling. Details in Appendix D.1.1.

Baselines and Evaluation. We compare against state-of-the-art diffusion-based world models: WPE [57] and IRA-Sim [86], which utilize AdaLN-Zero modulation [54] for the action embeddings, and Ctrl-World [14], which employs cross-attention with low-dimensional action tokens. For cross-embodiment methods, we adapt VLA architectures including unified action spaces [46], embodiment-aware encoders [6], and soft prompting [84]. All baselines use the same Wan-T2V-1.3B backbone, camera configurations, and frame schedules, but are trained for 20K steps (twice our method’s duration) to ensure a competitive comparison. Evaluation metrics include LPIPS [82] and PSNR [22], as they best align with human preferences for controllable world modeling [86].

Quantitative Analysis. We first evaluate in-distribution prediction (**Q1**) with a 7-DoF Franka-Panda (LIBERO-90) and a bi-manual dexterous robot (DexMimicGen), achieving a consistent average 11.4% improvement over baselines (Table 1). To assess generalization (**Q2**), we conduct zero-shot evaluation on LIBERO-10 and LIBERO-Spatial tasks, alongside few-shot finetuning on distinct hardware (Sawyer, IIWA, and UR5e) within the MimicGen-Robot, using the LIBERO-90 model. Across these novel trajectories, our pose-image conditioning yields a 16.8% average improvement over prior methods, confirming its superior generalization capability. Qualitative results in Appendix D.1.2.

¹The spatial-temporal VAE structure of Wan requires $4n + 1$ frames.

²The original implementation in Ctrl-World [14] uses end-effector position control, and all other baselines use end-effector velocity control. We found out that velocity control performs similar to position control on most datasets, while position control generalizes poorly in the new scene of LIBERO-Spatial where the position of the robot and objects are changed.

Method	Dataset	LIBERO-90		DexMimicGen		LIBERO-Long		LIBERO-Spatial		MimicGen-Robot	
		wrist-view	third-view	wrist-view	third-view	wrist-view	third-view	wrist-view	third-view	wrist-view	third-view
WPE		15.01 / 0.339	19.05 / 0.121	15.52 / 0.298	19.38 / 0.142	14.05 / 0.364	17.97 / 0.157	14.72 / 0.379	19.50 / 0.156	16.38 / 0.193	17.10 / 0.130
IRA-Sim		14.94 / 0.343	19.73 / 0.122	15.43 / 0.293	18.42 / 0.142	14.00 / 0.366	17.94 / 0.160	14.76 / 0.384	19.52 / 0.153	16.37 / 0.195	17.15 / 0.130
Crit-World vel. ²		15.54 / 0.318	20.10 / 0.109	15.56 / 0.276	19.19 / 0.135	14.63 / 0.345	18.26 / 0.139	15.50 / 0.361	19.76 / 0.137	16.80 / 0.171	17.43 / 0.124
Crit-World pos.		15.56 / 0.317	20.13 / 0.108	15.55 / 0.277	19.18 / 0.136	14.61 / 0.348	18.27 / 0.138	14.23 / 0.403	19.28 / 0.154	16.82 / 0.169	17.44 / 0.126
Ours		17.02 / 0.286	23.13 / 0.085	16.33 / 0.266	21.18 / 0.112	15.98 / 0.320	22.14 / 0.093	16.75 / 0.322	22.79 / 0.097	18.02 / 0.144	21.40 / 0.095
Rel. Improve.		9.4% / 9.8%	14.9% / 21.3 %	4.9% / 3.6%	10.4% / 17.0%	9.2% / 7.2%	21.2% / 32.6%	8.1% / 10.8%	15.3% / 29.2%	7.1% / 14.8%	22.7% / 23.4%

Table 1: **Results for single-embodiment action-conditioned world modeling.** We evaluate our pose-image conditioning against prior methods on diverse datasets with single embodiment. Each cell represents PSNR $\uparrow$ / LPIPS $\downarrow$ metrics averaged across the predicted frames, with the best being **bold** and the second best underline, and relative improvements computed against the second best number. We consistently outperform prior baselines with an average of 11.4% improvement for in-distribution data and 16.8% improvement in generalization settings.

Cross-Embodiment Modeling (Q3). To evaluate cross-embodiment world modeling, we train on a heterogeneous mixture of RoboCasa, MimicGen, and DexMimicGen, spanning action dimensions of 7 (fixed arm), 12 (mobile Panda), 14 (bi-manual parallel grippers), and 24 (bi-manual dexterous hands). As shown in Table 2, our method consistently outperforms prior approaches for cross-embodiment training. These results suggest that representing actions via robot pose images provides a scalable, embodiment-agnostic interface that may be of broader utility for future world-action models.

Method	Dataset	Mixture dataset	
		wrist-view	third-view
Unified action space		14.60 / 0.323	17.80 / 0.164
Embodiment-aware encoder		<u>14.68 / 0.321</u>	17.52 / 0.168
Soft prompt		14.66 / 0.323	<u>17.83 / 0.162</u>
Ours		15.11 / 0.308	19.44 / 0.131

Table 2: **Results for cross-embodiment modeling (PSNR $\uparrow$ / LPIPS $\downarrow$).** We use a mixture of trajectories from multiple robots with distinct action spaces. Our method consistently outperforms existing approaches.

5.2 World Action Planner

In this section, we evaluate our World Action Planner in three generalization scenarios: *compositional task generalization*, *new layout generalization*, and *zero-shot generalization*. Details in Appendix D.2.

Setup. We evaluate 12 tasks in the LIBERO [43] and Robosuite [87] environments, chosen for their modular design and object diversity. Success rates are measured across 50 trials per task. We employ Gemini 3.0 Flash [55] as our default VLM agent. For imitation learning baselines, we use SOTA VLA $\pi_{0.5}$ [27] and WAM cosmos-policy [35]. To isolate the impact of our action-conditioned world model, we compare against a *vision-language planner* baseline that directly executes actions from the VLM without further optimization and search. We further compare against prior world model-based policy enhancement methods where actions are sampled from policies rather than VLM agent. Specifically, we adapt SAILOR [29] with MPPI sampling and GPC-RANK [56] with best-of-N sampling for test time planning, using the same world model and VLM to provide imaginations and rewards.

5.2.1 Compositional Task Generalization

In real-world applications, robot policies are often required to generalize to compositional, long-horizon tasks. For instance, a robot may be trained to pick and place individual objects, but a task such as tidying a room requires it to execute multiple such placements in a continuous sequence. We adopt four tasks from LIBERO-Long, each composed of two tasks from the LIBERO-90 suite and *train our diffusion policy and world model on the LIBERO-90 dataset exclusively*. As the official checkpoints for $\pi_{0.5}$ and cosmos-policy do not include models trained on LIBERO-90, we fine-tune these baselines ourselves using the same data to ensure a fair comparison. Details in Appendix D.2.1.

When transferring to these newly composed tasks, we observe a distinct failure mode in end-to-end VLA models: while they often complete the first sub-task successfully, they stagnate immediately afterward with near no-op actions. This behavior stems from the policy’s inability to transition from the terminal state of the first sub-task to the starting configuration of the next, which is missing in the training demonstrations. Our World Action Planner overcomes this by reasoning with VLM agents; once the first sub-task is completed, the VLM agent proposes bridging actions to move the robot toward the subsequent target. During the local search process, we imagine policy rollouts following these interventions, confirming that the policy can successfully resume the next sub-task from the new state. As shown in Table 3, our method achieves significantly higher success rates than prior approaches.

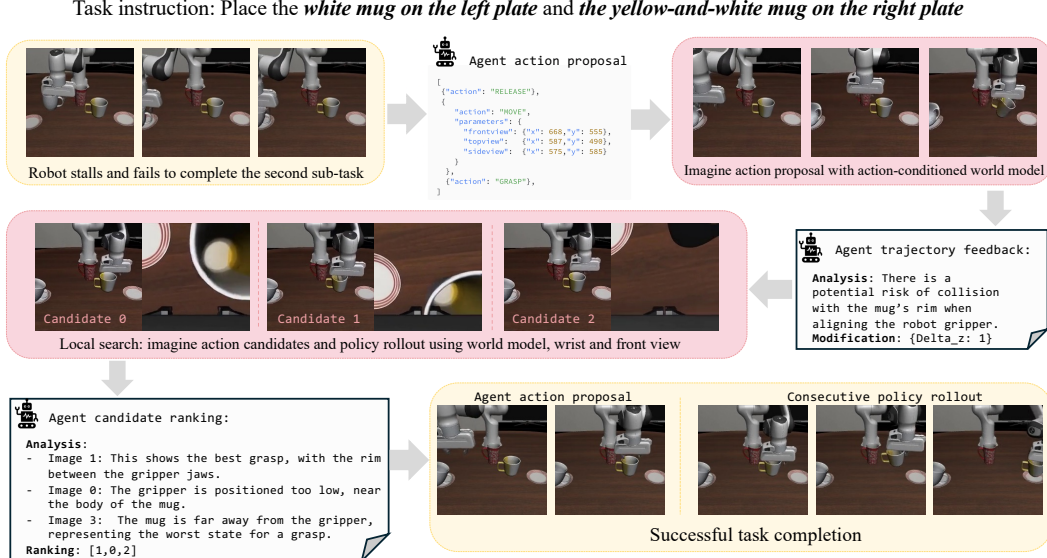


Figure 3: Illustration of world action planner in compositional task generalization

Method \ Task	PnP alphabet soup & tomato sauce	PnP white mug & yellow and white mug	PnP white mug & chocolate pudding	PnP alphabet soup & cream cheese box
$\pi_{0.5}$	4	0	0	0
cosmos-policy	0	0	0	0
SAILOR	18	0	8	2
GPC-RANK	10	0	0	0
Vision-language planner	56	28	46	32
World action planner	72	68	78	70

Table 3: Results for compositional task generalization. We evaluate on 4 tasks in the LIBERO-Long suite, each composed by 2 seen tasks in the LIBERO-90 dataset. Our method significantly outperforms prior baselines.

5.2.2 New Layout Generalization

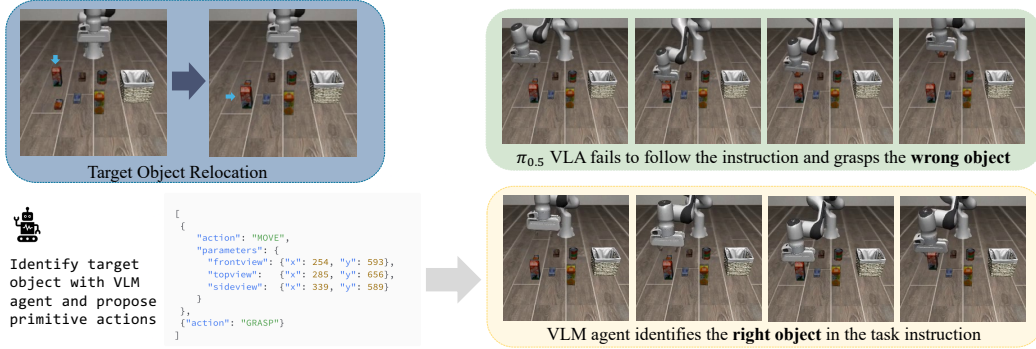


Figure 4: Illustration of new layout generalization

A generalizable robot must adapt to novel object configurations at test time, despite being trained on a limited distribution of layout samples. To evaluate this capability, we utilize six tasks from the LIBERO-Object suite involving pick-and-place operations, where we introduce significant layout shifts by varying the positions of both target and distractor objects during testing. For baselines, we employ the official LIBERO checkpoints for $\pi_{0.5}$ and cosmos-policy. We finetune the diffusion policy and world model—previously trained on LIBERO-90—using the official LIBERO-Object dataset, which contains demonstrations only in the *original layout*. To highlight our approach’s sample efficiency, we train our policy using only 5 *expert demonstrations* per task, whereas baselines utilize

the full dataset of over 40 demonstrations. For the world model, we augment these 5 demonstrations with 10 additional trajectories per task by applying Gaussian noise perturbations to the expert actions.

Method \ Task	PnP alphabet soup	PnP cream cheese	PnP salad dressing	PnP ketchup	PnP milk	PnP chocolate pudding
$\pi_{0.5}$	0	0	0	0	0	10
cosmos-policy	0	0	0	0	0	0
SAILOR	0	0	0	0	0	22
GPC-RANK	0	0	0	0	0	16
Vision-language planner	30	50	32	16	34	64
World action planner	88	86	90	66	84	78

Table 4: **Results for new layout generalization.** At test time we modify the layout and positions of the objects in the LIBERO-Object task suite. Our method consistently achieves high success rates. Details in Appendix D.2.2.

In our planner pipeline, the VLM agent identifies the target object within a novel layout and directs the robot to its proximity. Subsequently, our diffusion policy executes the grasp, followed by a second VLM-guided phase to navigate the gripper for placement. We observe that while the few-shot policy fails end-to-end due to navigation errors, it remains highly effective for the localized grasping manipulation required within our framework, even with modified layouts; the VLM agent successfully manages high-level identification and coarse navigation, allowing the policy to focus on fine-grained manipulation. This decoupling of high-level reasoning from low-level manipulation enables successful task completion with significantly less expert data than traditional end-to-end approaches. As shown in Table 4, our method achieves high success rates across all tasks. In contrast, generalist VLAs and WAMs fail to ground instructions to the modified scene, instead defaulting to motion priors from the training layout. This leads to persistent failure modes where baselines either grasp distractors or enter futile re-grasp cycles in empty space around the original target coordinates (Fig. 10, 11).

5.2.3 Zero-shot Generalization

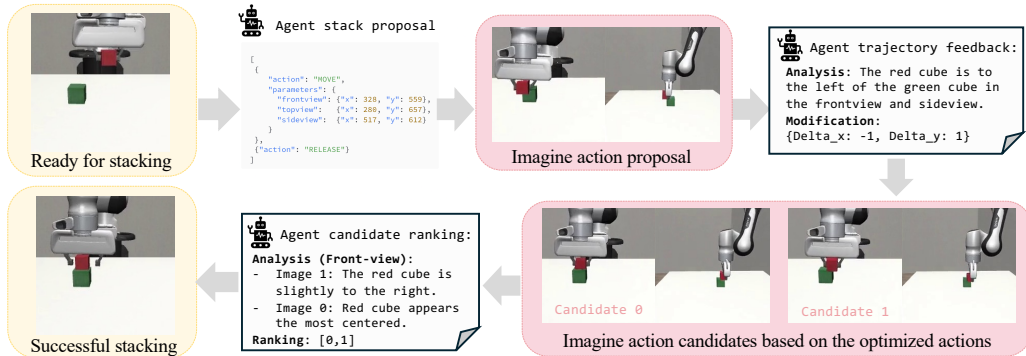


Figure 5: **Illustration of world action planner with zero-shot planning in StackCube.**

We evaluate the World Action Planner’s zero-shot capability to complete tasks in the absence of specialized policies or expert demonstrations. Using the PickPlaceCan and StackCube tasks from Robosuite, the VLM agent identifies target coordinates for grasping and placement, while the primitive gripper logic is hard-coded. We finetune the world model from Sec. 5.2.1 on 50 exploratory trajectories, similar to [29, 56]. As shown in Table 5, our approach achieves high success rates without a dedicated policy. Notably, the World Action Planner significantly outperforms a baseline vision-language planner, particularly during the precision-heavy stacking phase of StackCube. This performance gap highlights the necessity of refining coarse VLM action proposals through imagination using the action-conditioned world model. Details in Appendix D.2.3.

Method \ Task	PickPlaceCan	StackCube
Vision-language planner	58	22
World action planner	80	76

Table 5: **Results for zero-shot generalization.** We achieve high success rates even without any expert demonstrations.

5.2.4 Ablation Experiments

Here we ablate the components of our planner pipeline, with details in Appendix D.2.4. We first ablate the individual contributions of global optimization, local search, and policy rollout imagination.

The results are detailed in Table 9, where each component is essential for maximizing success as task complexity increases. The PnP cream cheese box requires specific gripper rotations that VLMs struggle to describe analytically but can accurately identify during local search by reasoning over imagined frames (Fig. 12). For the PnP mug task, imagining policy rollouts is critical to identify the correct state for the policy to complete the grasp, which the VLM cannot inherently predict (Fig. 13).

We further evaluate our planning pipeline against a naive Best-of-N VLM sampling baseline, with results detailed in Table 10. We observe that the actions proposed by the VLM often exhibit physical oversights and risks, such as collisions, across multiple samples. In contrast, our action optimization with agent feedback successfully identifies and corrects these potential risks (Fig. 14).

6 Conclusion and Limitations

In this work, we propose World Action Planner, a robot planning system to address novel tasks in generalization scenarios. A primary limitation is that our evaluations are conducted in simulation, while real robot experiments are left to future work. While the system is manually designed, it follows a principled coarse-to-fine optimization process, offering superior efficiency over large stochastic sampling used in traditional MPC methods. Finally, while world model imagination is time consuming, with wall clock time reported in Appendix E, inference acceleration and few step sampling techniques could be applied to further increase planning speed and real-time efficiency.

References

- [1] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári. Improved algorithms for linear stochastic bandits. *Advances in neural information processing systems*, 24, 2011. 21, 26
- [2] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025. 2, 16
- [3] A. Ali, J. Bai, M. Bala, Y. Balaji, A. Blakeman, T. Cai, J. Cao, T. Cao, E. Cha, Y.-W. Chao, et al. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025. 2, 16
- [4] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024. 16
- [5] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 5, 16
- [6] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 6
- [7] T. Bonnen, J. Malik, and A. Kanazawa. Human-level 3d shape perception emerges from multi-view learning. *arXiv preprint arXiv:2602.17650*, 2026. 3
- [8] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024. 16
- [9] B. Chen, T. Zhang, H. Geng, K. Song, C. Zhang, P. Li, W. T. Freeman, J. Malik, P. Abbeel, R. Tedrake, et al. Large video planner enables generalizable robot control. *arXiv preprint arXiv:2512.15840*, 2025. 3, 16, 17
- [10] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025. 4, 16, 28, 30

- [11] Y. Du, M. Yang, P. Florence, F. Xia, A. Wahid, B. Ichter, P. Sermanet, T. Yu, P. Abbeel, J. B. Tenenbaum, et al. Video language planning. *arXiv preprint arXiv:2310.10625*, 2023. 3, 16
- [12] J. Duan, W. Yuan, W. Pumacay, Y. R. Wang, K. Ehsani, D. Fox, and R. Krishna. Manipulate-anything: Automating real-world robots using vision-language models. *arXiv preprint arXiv:2406.18915*, 2024. 16
- [13] D. Ghosh, J. Rahme, A. Kumar, A. Zhang, R. P. Adams, and S. Levine. Why generalization in rl is difficult: Epistemic pomdps and implicit partial observability. *Advances in neural information processing systems*, 34:25502–25515, 2021. 17
- [14] Y. Guo, L. X. Shi, J. Chen, and C. Finn. Ctrl-world: A controllable generative world model for robot manipulation. *arXiv preprint arXiv:2510.10125*, 2025. 2, 3, 6, 16, 29
- [15] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019. 3, 16
- [16] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020. 16
- [17] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. 16
- [18] A. Hallak, D. Di Castro, and S. Mannor. Contextual markov decision processes. *arXiv preprint arXiv:1502.02259*, 2015. 5, 17
- [19] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023. 3, 6, 16
- [20] N. Hansen, X. Wang, and H. Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022. 16
- [21] X. He, H. Yu, S. Peng, D. Tan, Z. Shen, H. Bao, and X. Zhou. Matchanything: Universal cross-modality image matching with large-scale pre-training. *arXiv preprint arXiv:2501.07556*, 2025. 16
- [22] A. Hore and D. Ziou. Image quality metrics: Psnr vs. ssim. In *2010 20th international conference on pattern recognition*, pages 2366–2369. IEEE, 2010. 6
- [23] J. Hu, X. Chen, C. Jin, L. Li, and L. Wang. Near-optimal representation learning for linear bandits and linear rl. In *International Conference on Machine Learning*, pages 4349–4358. PMLR, 2021. 17
- [24] P. Hu, Y. Chen, and L. Huang. Nearly minimax optimal reinforcement learning with linear function approximation. In *International Conference on Machine Learning*, pages 8971–9019. PMLR, 2022. 18
- [25] Y. Hu, F. Lin, T. Zhang, L. Yi, and Y. Gao. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023. 2, 16
- [26] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023. 2, 16
- [27] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 3, 4, 7, 16, 17
- [28] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024. 36
- [29] A. K. Jain, V. Mohta, S. Kim, A. Bhardwaj, J. Ren, Y. Feng, S. Choudhury, and G. Swamy. A smooth sea never made a skilled sailor: Robust imitation via learning to search. *arXiv preprint arXiv:2506.05294*, 2025. 3, 6, 7, 9, 16, 29, 32, 33, 34

- [30] J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, Y. Fang, F. Hu, S. Huang, K. Kundalia, Y.-C. Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025. 16
- [31] E. Y.-T. Jia, W. Yuan, T. Shi, V. Guizilini, J. Mao, and Y. Wang. Dreamplan: Efficient reinforcement fine-tuning of vision-language planners via video world models. *arXiv preprint arXiv:2603.16860*, 2026. 2, 3, 16
- [32] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16923–16930. IEEE, 2025. 6
- [33] C. Jin, A. Krishnamurthy, M. Simchowitz, and T. Yu. Reward-free exploration for reinforcement learning. In *International Conference on Machine Learning*, pages 4870–4879. PMLR, 2020. 18, 19
- [34] C. Jin, Z. Yang, Z. Wang, and M. I. Jordan. Provably efficient reinforcement learning with linear function approximation. *Mathematics of Operations Research*, 48(3):1496–1521, 2023. 5, 20, 21, 26, 27
- [35] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. 3, 7, 16, 30
- [36] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 3, 16, 30, 31
- [37] F. Kong, X. Zhang, B. Wang, and S. Li. Improved regret bounds for linear adversarial mdps via linear optimization. *arXiv preprint arXiv:2302.06834*, 2023. 5, 21
- [38] J. Kwon, Y. Efroni, C. Caramanis, and S. Mannor. Reward-mixing mdps with few latent contexts are learnable. In *International Conference on Machine Learning*, pages 18057–18082. PMLR, 2023. 17
- [39] G. Li, Y. Wei, Y. Chi, Y. Gu, and Y. Chen. Sample complexity of asynchronous q-learning: Sharper analysis and variance reduction. *Advances in neural information processing systems*, 33:7031–7043, 2020. 5, 21
- [40] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026. 3, 16
- [41] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*, 2024. 36
- [42] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 3
- [43] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023. 6, 7
- [44] F. Liu, K. Fang, P. Abbeel, and S. Levine. Moka: Open-world robotic manipulation through mark-based visual prompting. *arXiv preprint arXiv:2403.03174*, 2024. 2, 3, 16
- [45] H. Liu, S. Yao, H. Chen, J. Gao, J. Mao, J.-B. Huang, and Y. Du. Simpack: Simulation-enabled action planning using vision-language models. *arXiv preprint arXiv:2512.05955*, 2025. 3
- [46] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024. 6
- [47] T. Lozano-Perez. Robot programming. *Proceedings of the IEEE*, 71(7):821–841, 2005. 2

- [48] R. Lu, G. Huang, and S. S. Du. On the power of multitask representation learning in linear mdp. *arXiv preprint arXiv:2106.08053*, 2021. 17
- [49] D. Malik, Y. Li, and P. Ravikumar. When is generalizable reinforcement learning tractable? *Advances in Neural Information Processing Systems*, 34:8032–8045, 2021. 6, 17
- [50] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. *arXiv preprint arXiv:2310.17596*, 2023. 6
- [51] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024. 6
- [52] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv preprint arXiv:2402.07872*, 2024. 16
- [53] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 5
- [54] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023. 6
- [55] S. Pichai, D. Hassabis, and K. Kavukcuoglu. A new era of intelligence with gemini 3. *Google Blog*, November, 18, 2025. 7
- [56] H. Qi, H. Yin, Y. Du, and H. Yang. Strengthening generative robot policies through predictive world modeling. *arXiv e-prints*, pages arXiv–2502, 2025. 3, 7, 9, 32, 33, 34
- [57] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang. Worldgym: World model as an environment for policy evaluation. *arXiv preprint arXiv:2506.00613*, 2025. 2, 3, 6, 16
- [58] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 5
- [59] N. Rajaraman, Y. Han, L. Yang, J. Liu, J. Jiao, and K. Ramchandran. On the value of interaction and function approximation in imitation learning. *Advances in Neural Information Processing Systems*, 34:1325–1336, 2021. 6, 17, 25
- [60] N. Rajaraman, L. Yang, J. Jiao, and K. Ramchandran. Toward the fundamental limits of imitation learning. *Advances in Neural Information Processing Systems*, 33:2914–2924, 2020. 17, 19, 20
- [61] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. 30
- [62] A. K. Sharma, Y. Sun, N. Lu, Y. Zhang, J. Liu, and S. Yang. World-gymnast: Training robots with reinforcement learning in a world model. *arXiv preprint arXiv:2602.02454*, 2026. 16
- [63] K. Song, B. Chen, M. Simchowitz, Y. Du, R. Tedrake, and V. Sitzmann. History-guided video diffusion. *arXiv preprint arXiv:2502.06764*, 2025. 3, 28
- [64] W. Sun, N. Jiang, A. Krishnamurthy, A. Agarwal, and J. Langford. Model-based rl in contextual decision processes: Pac bounds and exponential improvements over model-free approaches. In *Conference on learning theory*, pages 2898–2933. PMLR, 2019. 5, 17
- [65] Z. Tan, S. Liu, X. Yang, Q. Xue, and X. Wang. Ominicontrol: Minimal and universal control for diffusion transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 14940–14950, 2025. 3

- [66] G. R. Team, K. Choromanski, C. Devin, Y. Du, D. Dwibedi, R. Gao, A. Jindal, T. Kipf, S. Kirmani, I. Leal, et al. Evaluating gemini robotics policies in a veo world simulator. *arXiv preprint arXiv:2512.10675*, 2025. 2, 16
- [67] R. A. Team. Causal video models are data-efficient robot policy learners. *Rhoda AI Blog*, 2026. 3
- [68] A. J. Wagenmaker, Y. Chen, M. Simchowitz, S. Du, and K. Jamieson. Reward-free rl is no harder than reward-aware rl in linear markov decision processes. In *International Conference on Machine Learning*, pages 22430–22456. PMLR, 2022. 21
- [69] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 2, 6
- [70] Y. Wang, C. Wen, H. Guo, S. Peng, M. Qin, H. Bao, X. Zhou, and R. Hu. Precise action-to-video generation through visual action prompts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12713–12724, 2025. 2, 16
- [71] M. Weltevrede, C. Horsch, M. T. Spaan, and W. Böhmer. Exploration implies data augmentation: Reachability and generalisation in contextual mdps. *arXiv preprint arXiv:2410.03565*, 2024. 17
- [72] T. Wiedemer, Y. Li, P. Vicol, S. S. Gu, N. Matarese, K. Swersky, B. Kim, P. Jaini, and R. Geirhos. Video models are zero-shot learners and reasoners. *arXiv preprint arXiv:2509.20328*, 2025. 28
- [73] G. Williams, A. Aldrich, and E. A. Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017. 6
- [74] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long. ivideoqpt: Interactive videoqpts are scalable world models. *Advances in Neural Information Processing Systems*, 37:68082–68119, 2024. 16
- [75] S. Yang, Y. Du, K. Ghasemipour, J. Tompson, L. Kaelbling, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023. 2
- [76] Z. Yang, J. Teng, W. Zheng, M. Ding, S. Huang, J. Xu, Y. Yang, W. Hong, X. Zhang, G. Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024. 2
- [77] H. Ye, X. Chen, L. Wang, and S. S. Du. On the power of pre-training for generalization in rl: provable benefits and hardness. In *International Conference on Machine Learning*, pages 39770–39800. PMLR, 2023. 17
- [78] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 3, 4, 16
- [79] K. Young, A. Ramesh, L. Kirsch, and J. Schmidhuber. The benefits of model-based generalization in reinforcement learning. *arXiv preprint arXiv:2211.02222*, 2022. 17
- [80] M. Yuksekogonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024. 4
- [81] L. Zhang, A. Rao, and M. Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3836–3847, 2023. 16
- [82] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. 6
- [83] X. Zhang, H. Lin, H. Ye, J. Zou, J. Ma, Y. Liang, and Y. Du. Inference-time scaling of diffusion models through classical search. *arXiv preprint arXiv:2505.23614*, 2025. 28

- [84] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025. 6
- [85] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024. 5, 16
- [86] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong. Irasim: A fine-grained world model for robot manipulation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9834–9844, 2025. 2, 3, 6, 16
- [87] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 7

Appendix Overview. First, we provide a comprehensive review of related works in Appendix A. Proof of theoretical results is provided in Appendix B. Then we provide the details of our method in Appendix C and details of the experiments in Appendix D. We report the wall clock time of our world model in Appendix E.

A Additional Related Works

Controllable World Models. To generate and predict future states given control inputs, foundational video generative models typically condition on textual and visual descriptions [3, 2]. To facilitate more fine-grained interaction, these models can also condition on specific control signals, such as movements in games [17, 4] or latent action codes [8]. While most research focuses on predicting future pixel frames, latent world models instead predict state features [5, 85] obtained through unsupervised learning. To apply such controllable generation to robot manipulation, most existing works condition on low-dimensional action vectors through AdaLN-Zero modulation [57, 86] in diffusion models, or interleave low-dimensional action tokens and visual state tokens in auto-regressive models [74, 5]. Recently, [14] proposed cross-attention between frames and action tokens for improved control, while adopting multi-view prediction to enhance consistency. Furthermore, [66] demonstrated that pose-conditioned robot world models can successfully simulate OOD scenarios and unsafe executions, thereby serving as a tool to evaluate robot policies. [70] also adopted pose-image conditioning via the ControlNet [81] mechanism; however, they extract the ground truth robot poses from future frames using image matching [21], while we compute the poses using forward dynamics to imagine different actions during test time planning. Concurrently, [31] proposed rendering future arm images corresponding to actions as direct inputs to the video diffusion model.

World Models for Decision Making. Prior works have leveraged world models for reinforcement learning to reduce expensive real world interactions. The Dreamer series, for instance, optimizes policy returns using value functions and a latent dynamics model via policy gradients [15, 16] or actor-critic methods [17]. Recent works [62, 31] have conducted reinforcement learning (RL) on generalist VLAs and VLM planners, respectively, demonstrating that RL within world models can enhance performance on physical robots. To plan using world models, MPC-based methods [20, 19, 29] utilize the MPPI algorithm for exploration and update the sampling distribution based on Q-functions computed via the dynamics model. On the other hand, video planners [11, 30, 9] generate synthetic task completion videos using video generation models, and then extract the control actions using vision-based inverse-dynamics models or pose estimation techniques. In contrast to these prior works, we are the first to combine agentic reasoning with world model imagination for test-time planning in new environments.

VLMs for Robot Decision Making. To leverage the extensive world knowledge and reasoning capabilities of VLMs, the majority of existing works utilize these models for high-level task planning [12, 44, 26, 25]. However, to ground executable actions in the physical world, these methods typically rely on manually engineered or sampling-based manipulation techniques to generate low-level actions—such as grasp synthesis [44, 12], volumetric value maps [26], motion arrows [52], or teleoperation and kinesthetic teaching [25]. In contrast, our method leverages highly-expressive generative policies [10] as modular tools to execute atomic skills. By grounding the low-level actions proposed by the VLM through our fine-grained, pose-image conditioned world model, we enable the VLM to synthesize and reason directly over low-level executable actions.

VLMs and World Models for Imitation Learning Policies. Recent attempts have adopted pretrained VLMs or video world models as backbones for imitation learning policies, such as vision-language-action models (VLA) [27, 36] and world-action-models (WAM) [78, 35, 40]. These methods finetune action heads on top of the VLM or video generation model backbone to generate future actions, where OpenVLA [36] adopts auto-regressive action generation and $\pi_{0.5}$ [27] further incorporates diffusion-based action expert that denoises the action chunk. To adapt video generation models for policies, [78, 35, 40] first generates future frames and then generates the actions auto-regressively with the action head. However, these approaches often necessitate substantial quantities of high-quality action-labeled demonstration data, and their generalization performance can remain constrained, as observed in our experimental evaluation.

B Theoretical Insights

Here we provide the theoretical results which demonstrates the benefits of model-based learning with exploration over imitation learning [60, 59] in multi-task generalization settings, where different tasks share the same dynamics but have different rewards [38]. We focus on tabular MDPs and representation learning with linear function approximation. High level speaking, we show that model-based exploration can leverage data collected across all tasks, while imitation learning can only learn from the corresponding task expert, thus suffering from sub-optimal sample complexity.

B.1 Literature Overview

B.1.1 Separation Between Model-Based and Model-Free Learning

Prior research has demonstrated that model-free methods can exhibit exponentially worse sample complexity compared to model-based methods [64] for specific classes of MDPs. However, these findings typically rely on strong structural assumptions regarding the model class; in the standard tabular setting, there is no information-theoretic gap between model-based and model-free approaches [64, 79]. In our analysis, we show that within the tabular setting and under linear function approximation, multi-task imitation learning can be significantly more challenging than model-based planning. This stands in contrast to single-task settings, where imitation learning is generally more sample-efficient.

B.1.2 Generalization in Reinforcement Learning

In task-agnostic settings, where the agent lacks knowledge of the test environment’s context and is evaluated on its *average performance* across a family of MDPs, [49] established that MDPs must share certain similarities for a policy to generalize effectively. Furthermore, [77] derived average optimality gaps comparable to those in supervised learning without online interaction, and demonstrating that pretraining can significantly improve sample complexity when subsequent online interactions are permitted in the test environment. Also, [13, 49, 77] have shown that such task agnostic policies can not achieve instance optimality in new environment without further interaction or history dependence. In our framework, we assume the context of each environment is provided to the learner, and we aim for *instance optimality* without the need for online interactions during testing. Regarding multi-task representation learning, [48, 23] showed that learning a shared representation across tasks can improve sample complexity. We further assume that task-dependent context features are known to the learner and use them to construct context-dependent policies. Our setting is more aligned with modern robotic practices where task instructions serve as inputs to a generalist policy, and we derive representations for the tasks using pretrained models [9, 27].

B.2 Setup

We consider multi-task learning as contextual MDPs [18, 71] where MDPs with different contexts have shared dynamics and different rewards. Each MDP is described by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, r, \rho, H)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and H is the horizon. The initial state s_1 is drawn from the initial state distribution $\rho \in \Delta(\mathcal{S})$, and the state transition is governed by the transition probability $\mathbb{P}$. At each time step $h \in [H]$, the agent chooses action $a_h \in \mathcal{A}$ in state s_h according to policy $\pi(s_h) \in \Delta(\mathcal{A})$, receives reward $r(s_h, a_h)$ and transitions to the next state according to the transition kernel $\mathbb{P}(s_{h+1}|s_h, a_h)$. the episode terminates after H steps. Define the state-action value function (Q-function) and the state value function as the expected cumulative reward of π :

$$Q_h^\pi(s, a) = \mathbb{E} \left[\sum_{h'=h}^H r_{h'}(s_{h'}, a_{h'}) | s_h = s, a_h = a, \pi \right]$$

$$V_h^\pi(s) = \mathbb{E} \left[\sum_{h'=h}^H r_{h'}(s_{h'}, a_{h'}) | s_h = s, \pi \right].$$

which satisfies the Bellman function:

$$Q_h^\pi(s_h, a_h) = r(s_h, a_h) + \sum_{s_{h+1}} \mathbb{P}(s_{h+1}|s_h, a_h) V_{h+1}^\pi(s_{h+1}), \quad V_h^\pi(s_h) = \mathbb{E}_{a_h \sim \pi(s_h)} [Q_h(s_h, a_h)].$$

The goal of the agent is to achieve maximum expected cumulative reward $\mathbb{E}_{s_1 \sim \rho} V_1^\pi(s_1)$. Since the action space and episode length is finite, there always exist an optimal Q-function $Q_h^*(s, a) = \sup_\pi Q_h^\pi(s, a)$ and value function $V_h^*(s) = \sup_\pi V_h^\pi(s)$ for all s, a that satisfies the Bellman optimality equation:

$$Q_h^*(s_h, a_h) = r(s_h, a_h) + \sum_{s_{h+1}} \mathbb{P}(s_{h+1} | s_h, a_h) V_{h+1}^*(s_{h+1}), \quad V_h^*(s_h) = \operatorname{argmax}_{a \in \mathcal{A}} Q_h^*(s_h, a).$$

and there exists a deterministic policy $\pi^*(s_h) = \delta(\operatorname{argmax}_a Q_h^*(s_h, a))$ that is optimal with $Q_h^{\pi^*} = Q_h^*$. When the policy is deterministic, we abuse the notation and denote $a = \pi(s)$.

In the multi-task setting, we have a set of contexts (tasks) $\mathcal{C}$, with different reward functions $\{r_c : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1], c \in \mathcal{C}\}$, and the MDP associated with context c is $\mathcal{M}_c = (\mathcal{S}, \mathcal{A}, \mathbb{P}, r_c, \rho, H)$. We assume that the context c is determined before the episode and revealed to the agent. We then denote the policy and value functions given context c as $\pi_c(s)$ and $Q_h^\pi(s, a; c)$, $V_h^\pi(s; c)$ respectively. Since the context is known to the agent, we aim to obtain a context dependent policy $\pi_c(s)$ that maximizes $\mathbb{E}_{s_1 \sim \rho} V_1^\pi(s_1; c)$ for all context $c \in \mathcal{C}$. When rewards for different contexts share the same function approximation, we abuse notation and write $r(s, a; c) = r_c(s, a)$; similarly when policies for different contexts are from the same class, we write $\pi(s; c) = \pi_c(s)$.

B.2.1 Interaction Protocol

In this work we investigate the generalization task, where the agent is able to explore the environment and collect data during training time, with any context selected by the agent. At test time, the agent is evaluated by the performance of its policy under a randomly selected context. For the imitation learning setup, we further assume the existence of expert policies to collect demonstrations for any given context. Denote the total rounds of episode during the training phase as K , we specify the data collection and learning protocol for the imitation learning agent and model-based agent as below.

Imitation Learning Agent. At each episode $k \in [K]$, the imitation learning agent can select any context $c_k \in \mathcal{C}$ and query the deterministic expert policy $\pi_{c_k}^*$ to rollout in the environment $\mathcal{M}_{c_k}$ and collect data $\tau_{k, c_k} = \{(s_h^k, a_h^k)\}_{h=1}^H$, where $a_h^k = \pi_{c_k}^*(s_h^k)$. We denote the total data collected as $\mathcal{D} = \{\tau_{k, c_k}\}_{k=1}^K$. After the data collection phase, the learner outputs a context dependent policy $\hat{\pi} = \{\hat{\pi}_c, c \in \mathcal{C}\}$ minimizing the empirical risk of π with respect to $\mathcal{D}$ in a policy class Π :

$$\hat{\pi} = \operatorname{argmin}_{\pi \in \Pi} \Pr_{(s, a, c) \sim \mathcal{D}} (\pi_c(s) \neq a),$$

where we break ties arbitrarily.

Model-based Agent. At each episode $k \in [K]$, the agent can explore the select any $c_k \in \mathcal{C}$ and any policy π_k to collect trajectory $\tau_{k, c_k} = \{(s_h^k, a_h^k, r_h^k)\}_{h=1}^H$ in $\mathcal{M}_{c_k}$, where $a_h^k \sim \pi_k(s_h^k)$ and $r_h^k \sim r_c(s_h^k, a_h^k)$. After collecting the dataset $\mathcal{D} = \{\tau_{k, c_k}\}_{k=1}^K$, the learner outputs an estimate of the model $\hat{\mathbb{P}}$, and if the reward is unknown, an estimate of the reward $\hat{r} = \{\hat{r}_c, c \in \mathcal{C}\}$. At test time, given the random context c , the agent adopts a planner that outputs the optimal policy $\hat{\pi}_c$ with estimated MDP $\hat{\mathcal{M}}_c = (\mathcal{S}, \mathcal{A}, \hat{\mathbb{P}}, \hat{r}_c, \rho, H)$.

Suboptimality Gap. We define the suboptimality gap of the context dependent policy class $\pi = \{\pi_c, c \in \mathcal{C}\}$ returned by the agent as:

$$\mathbb{E}_{c \sim \text{Unif}(\mathcal{C}), s_1 \sim \rho} [V^*(s_1; c) - V^{\pi_c}(s_1; c)]. \quad (1)$$

B.3 Tabular MDPs

Here we consider tabular MDPs with finite state and action spaces and finite context set. We assume the reward function for each context is known to the agent and the transition is unknown [24]. We denote the number of states, actions and contexts as $|\mathcal{S}|$, $|\mathcal{A}|$ and $|\mathcal{C}|$.

B.3.1 Model-based Planning Agent

Here we prove Part (i) of Theorem 1. The result for the model-based exploration agent when generalizing to new rewards comes immediately with the reward-free exploration result of [33].

Theorem 4 (Theorem 3.1 of [33]). *There exists a reward-free model-based exploration algorithm, for any probability $\delta > 0$, with probability at least $1 - \delta$, outputs the model estimation $\hat{\mathbb{P}}$ such that for an arbitrary number of given reward functions r_c , the suboptimality gap of the optimal policy $\hat{\pi}_c$ solved from the estimated MDP $\widehat{\mathcal{M}}_c = (\mathcal{S}, \mathcal{A}, \hat{\mathbb{P}}, r_c, \rho, H)$ is smaller than ϵ . The total episodes of data collected in the exploration phase is $\tilde{\mathcal{O}}\left(\frac{H^5 |\mathcal{S}|^2 |\mathcal{A}|}{\epsilon^2}\right)$.*

Using the theorem above, we immediately obtain the $\mathcal{O}\left(\frac{1}{\sqrt{K}}\right)$ suboptimality gap, where $\mathcal{O}$ hides polynomial terms in H , $|\mathcal{S}|$ and $|\mathcal{A}|$. Note that this bound is independent of the context set size $|\mathcal{C}|$ since the result applies to any given reward function uniformly.

B.3.2 Imitation Learning Agent

here we provide the hardness result for imitation learning in Part ii of Theorem 1. High level speaking, since the expert policies are different at different tasks, the average number of demonstrations for each task is thus $\frac{K}{|\mathcal{C}|}$, and the suboptimality gap is $\Omega\left(\frac{1}{K}\right)$ following the $\Omega(1/N)$ lower bound in [60].

We design the hard MDP following [60, Sec. A.4.1]. For each state $s \in \mathcal{S}$, it is an absorbing state where $\mathbb{P}(\cdot|s, a) = \delta(s)$, $\forall a \in \mathcal{A}$. Denote $\xi = \frac{1}{N+1}$, where $N = \frac{K}{|\mathcal{C}|}$ is the average number of episodes in the dataset for each context. We define the initial state distribution $\rho = \{\xi, \xi, \dots, 1 - (|\mathcal{S}| - 1)\xi\}$.

We define the class of all context dependent deterministic policies as Π_{det} . We first sample the expert policy for each context uniformly and independently from Π_{det} , that is for each $s \in \mathcal{S}$, $c \in \mathcal{C}$, the optimal action $\pi_c^*(s)$ is uniformly sampled from $\mathcal{A}$. Given π_c^* , we define the reward function $r_c(\pi_c^*)$ where at each state, choosing the expert's action $\pi_c^*(s)$ receives reward 1 and otherwise 0. We denote the MDP with the transition and initial state distribution constructed above and the reward function $r_c(\pi_c^*)$ as $\mathcal{M}_c(\pi_c^*)$. It is obvious from the construction that the value of π_c^* in $\mathcal{M}_c(\pi_c^*)$ is H for any initial state s_1 , since it receives a reward of 1 at every step.

Define the set of deterministic policies $\pi = \{\pi_c, c \in \mathcal{C}\}$ that agrees with the expert policy in dataset D as:

$$\Pi(\mathcal{D}) = \left\{ \pi \in \Pi_{\text{det}}^{|\mathcal{C}|} : \pi_{c_k}(s_h^k) = a_h^k, \forall k \in [K], h \in [H] \right\}.$$

where we recall $a_h^k = \pi_{c_k}^*(s_h^k)$ in the expert dataset. We also denote

$$\mathcal{K}_c = \{k = 1, 2, \dots, K : c_k = c\}, \quad \mathcal{D}_c = \{\tau_{c_k, k}, k \in \mathcal{K}_c\}.$$

as the subset of episode indices and episodes the context of which is c . We then denote the class of deterministic policies conditioning on context c that agrees with the expert π_c^* in dataset D_c as:

$$\Pi_c(\mathcal{D}_c) = \left\{ \pi \in \Pi_{\text{det}} : \pi_c(s_h^k) = a_h^k, k \in \mathcal{K}_c \right\}.$$

We have the conditional distribution of $\pi^* = \{\pi_c^*, c \in \mathcal{C}\}$ conditioned on $\mathcal{D}$ is uniform in $\Pi(\mathcal{D})$, with the marginal conditional distribution of π_c^* being uniform in $\Pi_c(\mathcal{D}_c)$. Thus, the Bayes suboptimality gap of the output policy $\hat{\pi} = \{\hat{\pi}_c, c \in \mathcal{C}\}$ under the constructed MDP class $\mathcal{M}(\pi^*)$ can be written as:

$$\begin{aligned} & \mathbb{E}_{\pi^* \sim \text{Unif}(\Pi(\mathcal{D}))} \left[\mathbb{E}_{c \in \text{Unif}(\mathcal{C})} \left[\mathbb{E}_{s_1 \sim \rho} \left[V_{\mathcal{M}_c(\pi_c^*)}^*(s_1; c) - V_{\mathcal{M}_c(\pi_c^*)}^{\hat{\pi}_c}(s_1; c) \right] \right] \right] \\ &= \mathbb{E}_{c \in \text{Unif}(\mathcal{C})} \left[\mathbb{E}_{\pi_c^* \sim \text{Unif}(\Pi_c(\mathcal{D}_c))} \left[\mathbb{E}_{s_1 \sim \rho} \left[H - V_{\mathcal{M}_c(\pi_c^*)}^{\hat{\pi}_c}(s_1; c) \right] \right] \right] \end{aligned} \quad (2)$$

which is because the value of π_c^* in our constructed MDP $\mathcal{M}_c(\pi_c^*)$ is H ; and each entry π_c^* of π^* is independently uniform distributed in $\Pi_c(\mathcal{D}_c)$ so we apply Fubini's Theorem to exchange the expectations.

We denote the set of initial states visited in episodes with context c as

$$\mathcal{S}_1(\mathcal{D}_c) = \{s \in \mathcal{S} : \exists k \in \mathcal{K}_c, s_1^k = s\}.$$

Recall that in our MDP design each state is an absorbing state, so $\mathcal{S}_1(\mathcal{D}_c)$ is also the set of total states visited under context c in $\mathcal{D}$. Intuitively speaking, in context c , if $s_1 \sim \rho$ is from a unvisited state outside of $\mathcal{S}_1(\mathcal{D}_c)$, then the estimated policy $\hat{\pi}$ will suffer from constant errors. This is because the potential distribution of optimal action $\pi_c^*(s_1)$ given $\mathcal{D}$ is distributed uniformly in $\mathcal{A}$, independent of the data from other contexts, thus the agent can not guess the right action with high probability. Given context $c \in \mathcal{C}$, we have the following Lemma from [60].

Lemma 1 (Lemma A.19 of [60]). *Given context $c \in \mathcal{C}$ and dataset $\mathcal{D}$, we have the Bayes suboptimality of the imitation policy $\hat{\pi}_c$ lower bounded as:*

$$\mathbb{E}_{\pi_c^* \sim \text{Unif}(\Pi(\mathcal{D}))} \left[\mathbb{E}_{s_1 \sim \rho} \left[H - V_{\mathcal{M}_c(\pi_c^*)}^{\hat{\pi}_c}(s_1; c) \right] \right] \geq H \left(1 - \frac{1}{|\mathcal{A}|} \right) (1 - \rho(\mathcal{S}_1(\mathcal{D}_c)))$$

where $\rho(\mathcal{S}_1(\mathcal{D}_c)) = \sum_{s \in \mathcal{S}_1(\mathcal{D}_c)} \rho(s)$ is the total measure of all initial states visited in $\mathcal{D}_c$.

Next, we upper bound the average of $\sum_c \rho(\mathcal{S}_c(\mathcal{D}))$ for any dataset $\mathcal{D}$ consisting of K trajectories.

Lemma 2. *For any dataset $\mathcal{D}$ with K trajectories $\mathcal{D} = \{\tau_{k, c_k}\}_{k=1}^K$, we have*

$$\mathbb{E}_{c \sim \text{Unif}(\mathcal{C})} [1 - \rho(\mathcal{S}_c(\mathcal{D}))] \geq \frac{|\mathcal{S}| - 1}{e \left(\frac{K}{|\mathcal{C}|} + 1 \right)}$$

Proof. Recall the design of ρ as $\rho = \{\xi, \xi, \dots, 1 - (|\mathcal{S}| - 1)\xi\}$, where $\xi = \frac{1}{\bar{N}+1}$, $\bar{N} = \frac{K}{|\mathcal{C}|}$. Since for every episode index $k \in \mathcal{K}_c$, the initial state s_1^k is distributed independently according to ρ , we have:

$$\begin{aligned} 1 - \rho(\mathcal{S}_c(\mathcal{D})) &= \sum_{s \in \mathcal{S}} \rho(s) (1 - \rho(s))^{|\mathcal{K}_c|} \\ &\geq \frac{|\mathcal{S}| - 1}{\bar{N} + 1} \left(1 - \frac{1}{\bar{N} + 1} \right)^{|\mathcal{K}_c|}. \end{aligned}$$

Thus, using the Arithmetic Mean-Geometric Mean Inequality, we obtain:

$$\begin{aligned} \mathbb{E}_{c \sim \text{Unif}(\mathcal{C})} [1 - \rho(\mathcal{S}_c(\mathcal{D}))] &\geq \frac{|\mathcal{S}| - 1}{\bar{N} + 1} \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \left(1 - \frac{1}{\bar{N} + 1} \right)^{|\mathcal{K}_c|} \\ &\geq \frac{|\mathcal{S}| - 1}{\bar{N} + 1} \left(\prod_{c \in \mathcal{C}} \left(1 - \frac{1}{\bar{N} + 1} \right)^{|\mathcal{K}_c|} \right)^{\frac{1}{|\mathcal{C}|}} \\ &= \frac{|\mathcal{S}| - 1}{\bar{N} + 1} \left(1 - \frac{1}{\bar{N} + 1} \right)^{\frac{K}{|\mathcal{C}|}} \\ &\geq \frac{|\mathcal{S}| - 1}{e \left(\frac{K}{|\mathcal{C}|} + 1 \right)}. \end{aligned}$$

□

Finally, we prove the main theorem.

Proof of Part ii, Theorem 1. Given dataset $\mathcal{D}$, to bound the worst-case suboptimality of $\hat{\pi}$, it suffices to bound the Bayes optimality bound under our joint distribution of expert policies $\pi^* \sim \text{Unif}(\Pi(\mathcal{D}))$ and constructed MDP instances $\mathcal{M}(\pi^*)$. Using (2), Lemma 1 and Lemma 2, we have:

$$\mathbb{E}_{\pi^* \sim \text{Unif}(\Pi(\mathcal{D}))} \left[\mathbb{E}_{c \in \text{Unif}(\mathcal{C})} \left[\mathbb{E}_{s_1 \sim \rho} \left[V_{\mathcal{M}_c(\pi_c^*)}^*(s_1; c) - V_{\mathcal{M}_c(\pi_c^*)}^{\hat{\pi}_c}(s_1; c) \right] \right] \right] \geq \Omega \left(\frac{|\mathcal{C}|}{K} \right).$$

Thus, there exists a worst case instance of $\pi^* = \{\pi_c^*, c \in \mathcal{C}\}$ and MDP $\mathcal{M}(\pi^*) = \{\mathcal{M}_c(\pi_c^*), c \in \mathcal{C}\}$ given dataset $\mathcal{D}$, under which the suboptimality of the policy given by the imitation learning agent is at least $\Omega \left(\frac{|\mathcal{C}|}{K} \right)$. □

B.4 Linear Function Approximation

In this section, we provide the results under linear function approximation. First, we provide the definition of linear MDP following [34]:

Definition 1 (Linear MDP [34]). MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, r, \rho, H)$ is called a linear MDP with feature map $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^d$, if there exist d (unknown) signed measures $\mu_h = (\mu_h^{(1)}, \mu_h^{(2)}, \dots, \mu_h^{(d)})$ over $\mathcal{S}$ such that the transition probability can be represented as linear functions

$$\mathbb{P}_h(\cdot | s, a) = \langle \phi(s, a), \mu_h(\cdot) \rangle,$$

and (unknown) vector $\theta_h \in \mathbb{R}^d$ such that the expectation of the random reward can be represented as a linear function

$$\mathbb{E}[r_h(s, a)] = \langle \phi(s, a), \theta \rangle, \quad 0 \leq r_h(s, a) \leq 1,$$

for all $(s, a) \in \mathcal{S} \times \mathcal{A}$. With out loss of generality, we assume $\|\phi(s, a)\| \leq 1$ for all (s, a) and $\max\{\|\mu_h(\mathcal{S})\|, \|\theta\|\} \leq \sqrt{d}$ for all h .

Given the above definition, we further explore the setting of learning the reward function for generalizing to new tasks with task features. Specifically, we assume that different reward vectors θ_c share a common feature map $\psi(c) \in \mathbb{R}^d$ known to the agent, with $\theta_c = \Theta\psi(c)$. Thus, by learning the transformation Θ using reward samples from source tasks, we can generalize to any new task using the representation ψ . We point out that learning such reward function with linear structure is non-trivial due to the random noise in the reward received [1].

Assumption 1 (Reward representation). There exists a feature map $\psi : \mathcal{C} \rightarrow \mathbb{R}^{d'}$ and (unknown) matrix $\Theta \in \mathbb{R}^{d \times d'}$, where the random reward for context c can be written as $\mathbb{E}[r_c(s, a)] = \phi(s, a)^\top \Theta \psi(c)$ with $0 \leq r_c(s, a) \leq 1$ and $\|\psi(c)\| \leq 1$, for all $(s, a, c) \in \mathcal{S} \times \mathcal{A} \times \mathcal{C}$. We assume the set $\{\psi(c), c \in \mathcal{C}\} \subset \mathbb{R}^d$ is compact and $\text{span}(\{\psi(c), c \in \mathcal{C}\}) = \mathbb{R}^d$.

To explore the feature set, we define the G-optimal design of the feature set as:

Theorem 5 (Kiefer–Wolfowitz). There exists a distribution $p_c \in \Delta(\mathcal{C})$, such that $|\text{Supp}(p_c)| \leq \frac{d'(d'+1)}{2}$ and $\max_{c \in \mathcal{C}} \|\psi(c)\|_{V^{-1}} = d'$, where V is the covariance matrix under p_c :

$$V = \sum_{c \in \mathcal{C}} p_c \psi(c) \psi(c)^\top.$$

We also assume that the MDP has well-conditioned covariates and there exists a *unknown* policy that can explore the h layer of the MDP, following standard practice [37, 39]

Assumption 2 (Exploratory assumption). There exists $\kappa > 0$, such that

$$\min_{h \in [H]} \sup_{\pi} \lambda_{\min}(\mathbb{E}_{\pi}[\phi(s_h, a_h) \phi(s_h, a_h)^\top]) \geq \kappa.$$

B.4.1 Model-based Planning Agent

In this section, we prove the suboptimality gap for the model-based planning agent under our linear function approximation. We first restate the result from [68] that enables sufficient exploration of the feature space, without knowledge of any exploratory policies:

Theorem 6 (Theorem 4, [68]). Fix $\gamma \in [0, 1]$, under Assumption 2. There exists an algorithm that collects observations $\mathcal{D} = \{(s_1^k, a_1^k), \dots, (s_H^k, a_H^k)\}_{k=1}^K$, such that with probability $1 - \delta$, we have for any h :

$$\lambda_{\min} \left(\sum_{k=1}^K \phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top \right) \geq \frac{\kappa}{\gamma^2}.$$

after running at most

$$K = \mathcal{O} \left(\frac{dH}{\kappa \gamma^2} \right)$$

where $\mathcal{O}$ hides polynomial dependencies on H and $\log \frac{1}{\delta}$.

Data Collection Protocol. We jointly explore the state-action feature space $\phi(s, a)$ and the context feature space $\psi(c)$ by combining our exploration algorithm with a G-optimal design. Specifically, for each context $c \in \text{Supp}(p_c)$, we execute the exploration procedure described in Theorem 6 for K_c episodes. We denote the dataset collected under context c as $\mathcal{D}_c$, and the corresponding episode indices as $\mathcal{K}_c = \{k \in [K] : c_k = c\}$. The total dataset across all contexts is defined as $\mathcal{D} = \cup_{c \in \mathcal{C}} \mathcal{D}_c$. To estimate the model parameters, we define the context-specific covariance matrices at step h as $\Lambda_h^c = \sum_{k \in \mathcal{K}_c} \phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top$, and the aggregate covariance across all episodes as $\Lambda_h = \sum_{k=1}^K \phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top + \mathbf{I}$.

Building on the high probability event in Theorem 6, we have the total number of episodes be bounded as $\mathcal{O}\left(\frac{\text{poly}(d, d', H)}{\kappa \gamma^2}\right)$:

Lemma 3. *For any $\delta > 0$, with probability $1 - \delta$, we have for each $c \in \text{Supp}(p_c)$ and $h \in [H]$, $\lambda_{\min}(\Lambda_h^c) \geq \frac{\kappa}{\gamma^2}$, and the total number of episodes be bounded as $K = \tilde{\mathcal{O}}\left(\frac{d(d')^2 H}{\kappa \gamma^2}\right)$.*

Proof. From Theorem 5, we have $|\text{Supp}(p_c)| = \mathcal{O}(d'^2)$. Thus, the procedure in Theorem 6 will be ran for at most $|\text{Supp}(p_c)|$ rounds, and the guarantee follows by applying a union bound of the event in Theorem 6 over all rounds. $\square$

Next, we bound the error of the estimated model:

Lemma 4. *We estimate the transition model as:*

$$\hat{\boldsymbol{\mu}}_h = \Lambda_h^{-1} \sum_{k=1}^K \phi(s_h^k, a_h^k) \delta(s_{h+1}^k)^\top.$$

We have the following error bound for any function $V : \mathcal{S} \rightarrow \mathbb{R}$ that is the optimal value function for a linear MDP: with probability at least $1 - \delta$, we have

$$\left| (\hat{\mathbb{P}}_h - \mathbb{P}_h) V(s, a) \right| \leq \tilde{\mathcal{O}} \left(\text{poly}(d, H) \sqrt{\frac{\gamma^2}{\kappa}} \right),$$

for any $(s, a) \in \mathcal{S} \times \mathcal{A}$, $h \in [H]$ and $0 < \delta < 1$.

Proof. We decompose the LHS as:

$$\begin{aligned} \left| (\hat{\mathbb{P}}_h - \mathbb{P}_h) V(s, a) \right| &= |\langle \phi(s, a), (\hat{\boldsymbol{\mu}}_h - \boldsymbol{\mu}_h) V(s, a) \rangle| \\ &\leq \|\phi(s, a)\|_{\Lambda_h^{-1}} \|(\hat{\boldsymbol{\mu}}_h - \boldsymbol{\mu}_h) V(s, a)\|_{\Lambda_h}. \end{aligned}$$

According to our least squares estimator, we have:

$$\begin{aligned} \hat{\boldsymbol{\mu}}_h - \boldsymbol{\mu}_h &= \Lambda_h^{-1} \sum_{k=1}^K \phi(s_h^k, a_h^k) \delta(s_{h+1}^k)^\top - \Lambda_h^{-1} \left(\sum_{k=1}^K \phi(s_h^k, a_h^k) \mathbb{P}(\cdot | s_h^k, a_h^k) + \boldsymbol{\mu}_h \right) \\ &= \Lambda_h^{-1} \sum_{k=1}^K \phi(s_h^k, a_h^k) \left(\delta(s_{h+1}^k)^\top - \mathbb{P}(\cdot | s_h^k, a_h^k) \right) - \Lambda_h^{-1} \boldsymbol{\mu}_h, \end{aligned}$$

which is because of our linear transition function:

$$\sum_{k=1}^K \phi(s_h^k, a_h^k) \mathbb{P}(\cdot | s_h^k, a_h^k) + \boldsymbol{\mu}_h = \sum_{k=1}^K \phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top \boldsymbol{\mu}_h + \boldsymbol{\mu}_h = \Lambda_h \boldsymbol{\mu}_h.$$

Thus, we have:

$$\|(\hat{\boldsymbol{\mu}}_h - \boldsymbol{\mu}_h) V(s, a)\|_{\Lambda_h} \leq \left\| \sum_{k=1}^K \phi(s_h^k, a_h^k) (V(s_{h+1}^k) - \mathbb{E}[V(s_{h+1}^k) | s_h^k, a_h^k]) \right\|_{\Lambda_h^{-1}} + \sqrt{d}.$$

Applying Lemma 7 and Lemma 8 with the covering parameter $\epsilon = 1/K$, we have:

$$\|(\hat{\boldsymbol{\mu}}_h - \boldsymbol{\mu}_h) V(s, a)\|_{\Lambda_h} \leq \text{poly} \left(d, H, \log K, \log \frac{1}{\delta} \right).$$

On the other hand, based the event in on Lemma 3, we have $\Lambda_h \succeq \sup_{c \in \text{Supp}(p_c)} \Lambda_h^c \succeq \frac{\kappa}{\gamma^2} \mathbf{I}$, thus we have $\|\phi(s, a)\|_{\Lambda_h^{-1}} \leq \sqrt{\frac{\gamma^2}{\kappa}}$. Also, we have $K = \tilde{\mathcal{O}} \left(\frac{dd'^2}{\kappa\gamma^2} \right)$. Putting together we prove our lemma. $\square$

Now, we move on to estimate the reward parameters Θ_h . Note that the reward mean can be written as:

$$\mathbb{E}[r_h(s, a; c)] = \phi(s, a)^\top \Theta_h \psi(c) = (\psi(c)^\top \otimes \phi(s, a)^\top) \text{vec}(\Theta_h)$$

Denote $\varphi(s, a, c)$ as $\psi(c) \otimes \phi(s, a)$ and $\text{vec}(\Theta_h) = \vartheta_h$, we have $\phi(s, a)^\top \Theta_h \psi(c) = \varphi(s, a, c)^\top \vartheta_h$. We have the following results for the reward estimate $\hat{r}$:

Lemma 5. *Denote*

$$V = \left(\mathbf{I} + \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) \varphi(s_h^k, a_h^k, c_k)^\top \right)^{-1},$$

then we estimate the $\hat{\vartheta}_h = \text{vec}(\Theta_h)$ using linear regression:

$$\hat{\vartheta}_h = V^{-1} \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) r_h(s_h^k, a_h^k, c_k).$$

We have with probability at least $1 - \delta$ for $0 < \delta < 1$, for all $(s, a, c) \in \mathcal{S} \times \mathcal{A} \times \mathcal{C}$,

$$|\hat{r}_h(s, a; c) - r_h(s, a; c)| \leq \tilde{\mathcal{O}} \left(\text{poly}(d, d') \sqrt{\frac{\gamma^2}{\kappa}} \right).$$

Proof. Using the linear regression estimator, we have:

$$\hat{\vartheta}_h - \vartheta_h = V^{-1} \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) (r_h(s_h^k, a_h^k, c_k) - \mathbb{E}[r_h(s_h^k, a_h^k, c_k)]) - V^{-1} \vartheta_h.$$

Thus, we can bound the estimation error of as:

$$\begin{aligned} & |\hat{r}_h(s_h, a_h, c) - r_h(s_h, a_h, c)| \\ &= \left| \left\langle \varphi(s, a), \hat{\vartheta}_h - \vartheta_h \right\rangle \right| \\ &\leq \|\varphi(s_h, a_h, c)\|_{V^{-1}} \left\| -\vartheta_h + \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) \eta_h^k \right\|_{V^{-1}} \\ &\leq \tilde{\mathcal{O}} \left(dd' + \sqrt{dd' + \log \frac{1}{\delta}} \right) \|\varphi(s_h, a_h, c)\|_{V^{-1}} \end{aligned}$$

where η_h^k is the noise of the random reward:

$$\eta_h^k = (r_h(s_h^k, a_h^k, c_k) - \mathbb{E}[r_h(s_h^k, a_h^k, c_k)]),$$

and we use Lemma 6 to bound $\left\| \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) \eta_h^k \right\|_{V^{-1}}$.

We now process the term $\|\varphi(s_h, a_h, c)\|_{V^{-1}}$. First, we lower bound V :

$$\begin{aligned}
& \sum_{k=1}^K \varphi(s_h^k, a_h^k, c_k) \varphi(s_h^k, a_h^k, c_k)^\top \\
&= \sum_{k=1}^K (\psi(c_k) \otimes \phi(s_h^k, a_h^k)) (\psi(c_k)^\top \otimes \phi(s_h^k, a_h^k)^\top) \\
&= \sum_{k=1}^K (\psi(c_k) \psi(c_k)^\top) \otimes (\phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top) \\
&= \sum_{c \in \text{Supp}(p_c)} (\psi(c) \psi(c)^\top) \otimes \left(\sum_{k \in \mathcal{K}_c} (\phi(s_h^k, a_h^k) \phi(s_h^k, a_h^k)^\top) \right) \\
&\succeq \left(\sum_{c \in \text{Supp}(p_c)} (\psi(c) \psi(c)^\top) \right) \otimes \frac{\kappa}{\gamma^2} \mathbf{I}
\end{aligned}$$

Denote $G = \sum_{c \in \text{Supp}(p_c)} (\psi(c) \psi(c)^\top)$, by the property of the G-optimal design p_c in Theorem 5, we have for any c ,

$$\|\psi(c)\|_{G^{-1}} \leq \left\| \psi(c) \left(\sum_{c \in \text{Supp}(p_c)} p_c (\psi(c) \psi(c)^\top) \right)^{-1/2} \right\|_2 \leq d'$$

since $G \succeq \sum_{c \in \text{Supp}(p_c)} p_c (\psi(c) \psi(c)^\top)$.

As a result, we have:

$$\begin{aligned}
& \|\varphi(s_h, a_h, c)\|_{V^{-1}} \\
&= \left\| (\psi(c) \otimes \phi(s_h, a_h)) V^{-1/2} \right\|_2 \\
&\leq \left\| (\psi(c) \otimes \phi(s_h, a_h)) \left(G \otimes \frac{\kappa}{\gamma^2} \mathbf{I} \right)^{-1/2} \right\|_2 \\
&\leq \sqrt{\frac{\gamma^2}{\kappa}} \left\| (\psi(c) G^{-1/2}) \otimes \phi(s_h, a_h) \right\|_2 \\
&= \sqrt{\frac{\gamma^2}{\kappa}} \|\psi(c)\|_{G^{-1}} \\
&\leq \sqrt{\frac{\gamma^2}{\kappa}} d'
\end{aligned}$$

Thus we have with probability at least $1 - \delta$, for any (s_h, a_h, c) :

$$|\widehat{r}_h(s_h, a_h, c) - r_h(s_h, a_h, c)| \leq \mathcal{O} \left(\sqrt{\frac{\gamma^2}{\kappa}} \text{poly}(d, d') \right).$$

□

Proof of Theorem 2. Finally, we bound the error of $\widehat{\pi}$ which is the optimal policy in the estimated MDP $\widehat{\mathcal{M}}$, against π^* which is the optimal policy of $\mathcal{M}$. For the value function of a policy π in $\mathcal{M}$ and $\widehat{\mathcal{M}}$, we denote them as V^π and $\widehat{V}^\pi$, respectively. We can decompose the optimality gap as:

$$V^* - V^{\widehat{\pi}} = (V^{\pi^*} - \widehat{V}^{\pi^*}) + (\widehat{V}^{\pi^*} - \widehat{V}^{\widehat{\pi}}) + (\widehat{V}^{\widehat{\pi}} - V^{\widehat{\pi}}) \leq (V^{\pi^*} - \widehat{V}^{\pi^*}) + (\widehat{V}^{\widehat{\pi}} - V^{\widehat{\pi}}).$$

Using the simulation lemma, we have:

$$\begin{aligned} V^{\pi^*} - \widehat{V}^{\pi^*} &= \mathbb{E}_{\widehat{\mathcal{M}}, \pi^*} \left[\sum_{h=1}^H (r_c(s_h, a_h) - \widehat{r}_c(s_h, a_h)) + (\mathbb{P}_h - \widehat{\mathbb{P}}_h) V^{\pi^*}(s_h, a_h) \right], \\ \widehat{V}^{\widehat{\pi}} - V^{\widehat{\pi}} &= \mathbb{E}_{\mathcal{M}, \widehat{\pi}} \left[\sum_{h=1}^H (\widehat{r}_c(s_h, a_h) - r_c(s_h, a_h)) + (\widehat{\mathbb{P}}_h - \mathbb{P}_h) \widehat{V}^{\widehat{\pi}}(s_h, a_h) \right]. \end{aligned}$$

Since the value functions $\widehat{V}^{\widehat{\pi}}$ and V^{π^*} are the optimal value functions of $\widehat{\mathcal{M}}$ and $\mathcal{M}$, invoking Lemma 4 and 5, we have for any $(s, a, c) \in \mathcal{S} \times \mathcal{A} \times \mathcal{C}$:

$$\begin{aligned} |\widehat{r}_c(s_h, a_h) - r_c(s_h, a_h)| &\leq \tilde{\mathcal{O}} \left(\sqrt{\frac{\gamma^2}{\kappa}} \text{poly}(d, d') \right), \\ \left| (\widehat{\mathbb{P}}_h - \mathbb{P}_h) V(s, a) \right| &\leq \tilde{\mathcal{O}} \left(\text{poly}(d, H) \sqrt{\frac{\gamma^2}{\kappa}} \right), \quad V \in \{\widehat{V}^{\widehat{\pi}}, V^*\}. \end{aligned}$$

Based on Lemma 3, we have $K = \tilde{\mathcal{O}} \left(\frac{dd'^2 H}{\kappa \gamma^2} \right)$, thus both $V^{\pi^*} - \widehat{V}^{\pi^*}$ and $\widehat{V}^{\widehat{\pi}} - V^{\widehat{\pi}}$ are bounded by $\tilde{\mathcal{O}} \left(\frac{\text{poly}(d, d', H)}{\kappa \sqrt{K}} \right)$. We then obtain the desired result of $V^* - V^{\widehat{\pi}} \leq \tilde{\mathcal{O}} \left(\frac{\text{poly}(d, d', H)}{\kappa \sqrt{K}} \right)$. $\square$

B.4.2 Imitation Learning Agent

Here we provide proof for Theorem 3.

Positive Result. In the single task setting, according to the properties of linear MDP, there exists $w_h^* \in \mathbb{R}^d$ such that $Q^*(s, a) = \langle \phi(s, a), w_h^* \rangle$, so the optimal policy can be represented as a linear classifier using Q^* : $\pi^*(s) = \arg\max_a Q^*(s, a) = \arg\max_a \langle \phi(s, a), w_h^* \rangle$. Following the results in [59], we can learn the optimal linear policy with suboptimality gap $\tilde{\mathcal{O}} \left(\frac{\text{poly}(d, H)}{K} \right)$ through imitation learning with empirical risk minimization (ERM).

Next, we will provide the proof for the hardness result in the multi-task setting. To prove that there does not exist $f \in \mathcal{F}$ with $\pi^*(s, a; c) = \arg\max_a f(\phi(s, a); \psi(c))$, we construct a set of context c such that they are the roots of $f(\phi(s, a_1); \psi(c)) - f(\phi(s, a_2); \psi(c))$ by showing the optimal action a changed from a_1 to a_2 when the context moves past c . By creating a large set of non-linear roots $\psi(c)$, we show that f must have non-linear and high-order dependency on $\psi(c)$. Notice that we do not require $Q^* \in \mathcal{F}$ for the optimal policy to be realizable, although the optimal Q-function $Q^*(s, a; c)$ also has complicated dependency on $\psi(c)$.

Here, we provide the construction of the hard instance. We construct our MDP with two levels $H = 2$ and feature dimension $d = 3$. We denote actions as $\mathcal{A} = \{1, 2, \dots, |\mathcal{A}|\}$. Now, we define the feature maps as follows. At the fixed initial state s_1 , the actions have features $\phi(s_1, 1) = [1, 0, 0]$, $\phi(s_1, 2) = [0, 1, 0]$ and for all actions $a = 3, \dots, |\mathcal{A}|$, we have $\phi(s_1, a) = [0, 0, 1]$. The reward at the first level is always 0. We have three states at $h = 1$, with $\mu(1) = [1, 0, 0]$, $\mu(2) = [0, 1, 0]$ and $\mu(3) = [0, 0, 1]$. So taking action 1 will move to state 1, taking action 2 will move to state 2 and taking action $a \geq 3$ will move to state 3. We design state 3 as an empty state, such that for all actions in state 3, the feature $\phi(3, a) = \mathbf{0}$ so the reward is always 0 in state 3. As a result, the value of state 3 is always 0, and the Q-function $Q(s_1, a)$ of all actions $a \geq 3$ is 0.

At the second level, for the actions $a = 1, 2, \dots, |\mathcal{A}|$, we design the feature set in states $s_2 = 1, 2$ to be $\phi(1, i) = \left[\cos \pi \frac{2a}{|\mathcal{A}|}, \sin \pi \frac{2a}{|\mathcal{A}|}, 0 \right]$, and $\phi(2, i) = \left[\cos \pi \frac{2a-1}{|\mathcal{A}|}, \sin \pi \frac{2a-1}{|\mathcal{A}|}, 0 \right]$.

For the context dependent reward, we define the continuous context set as $\mathcal{C} = [0, 1]$ so $0 < c < 1$, and the context feature as $\psi(c) = [\cos 2\pi c, \sin 2\pi c, 0]$. We simply set $\Theta = \mathbf{I}$. Thus, we have the reward at the second level defined as $r_2(1, a, c) = \cos \left(2\pi c - 2\pi \frac{a}{|\mathcal{A}|} \right)$, and $r_2(2, a, c) = \cos \left(2\pi c - \pi \frac{2a-1}{|\mathcal{A}|} \right)$. Suppose $2|\mathcal{A}|c \in [n_c, n_c + 1]$ where n_c is an integer, then at the second level the optimal value

function for the two states are:

$$\begin{aligned}
V^*(1, c) &= \operatorname{argmax}_i \cos \left(2\pi c - 2\pi \frac{i}{|\mathcal{A}|} \right) \\
&= \operatorname{argmax}_i \cos \left(\frac{\pi}{|\mathcal{A}|} (2|\mathcal{A}|c - 2i) \right) \\
&= \cos \left(2\pi c - \pi \frac{2\lfloor \frac{n_c+1}{2} \rfloor}{|\mathcal{A}|} \right) \\
V^*(2, c) &= \operatorname{argmax}_i \cos \left(2\pi c - \pi \frac{2i-1}{|\mathcal{A}|} \right) \\
&= \operatorname{argmax}_i \cos \left(\frac{\pi}{|\mathcal{A}|} (2|\mathcal{A}|c - (2i-1)) \right) \\
&= \cos \left(2\pi c - \pi \frac{2\lfloor \frac{n_c}{2} \rfloor + 1}{|\mathcal{A}|} \right)
\end{aligned}$$

Thus, the optimal action $\pi(s_1; c)$ is 1 if $V^*(1, c) > V^*(2, c)$ and 2 otherwise, since taking action 1 will move to state 1 and taking action 2 will move to state 2.

Now, assume that the optimal policy can be represented by a function

$$\pi^*(s_1; c) = \operatorname{argmax}_{a=1,2} f_a(\psi(c)),$$

where we assume f_a can be written as a polynomial in the features $\psi(c)$ with degree n :

$$f_a(\psi(c)) = \sum_{u+v+w \leq n} g_a^{u+v+w} \psi(c)_0^u \psi(c)_1^v \psi(c)_2^w = \sum_{u+v \leq n} g_a^{u,v} \sin^u 2\pi c \cos^v 2\pi c, a = 1, 2.$$

Consider $f(\psi(c)) = f_1(\psi(c)) - f_2(\psi(c))$ which is a n -degree polynomial with $\psi(c)$. Notice that at points $c_i = \frac{2i+1}{4|\mathcal{A}|}$ where $i = 0, 1, 2, \dots, 2|\mathcal{A}| - 1$, we have $V^*(2, c_i) = V^*(1, c_i)$. Also, for any small enough $\epsilon \geq 0$, we have $(V^*(2, c_i - \epsilon) - V^*(1, c_i - \epsilon))(V^*(2, c_i + \epsilon) - V^*(1, c_i + \epsilon)) < 0$. That is, the optimal action of $\pi^*(s_1, c)$ is being changed when moving past c_i , so c_i is a root of $f(\psi(c))$. Since $f(\psi(c))$ have at most $2n$ roots following Lemma 9 and we have constructed $2|\mathcal{A}|$ roots c_i , we conclude that its degree n must be at least $|\mathcal{A}|$.

B.4.3 Technical Lemmas

Here we provide the concentration inequality for vector-valued martingales, which will be used in our analysis of the model-based method in the linear function approximation setting.

Lemma 6 (Hoeffding concentration inequality for vector-valued martingales, Theorem 1 in [1]). *Let $\{\eta_\tau\}_{\tau=1}^\infty$ be a stochastic process with corresponding filtration $\{\mathcal{F}_\tau\}_{\tau=1}^\infty$. Let $\eta_\tau | \mathcal{F}_{\tau-1}$ be zero mean and σ -subGaussian. Let $\{\phi_\tau\}_{\tau=1}^\infty$ be a $\mathbb{R}^d$ -valued stochastic process with $\phi_\tau \in \mathcal{F}_{\tau-1}$, and $\|\phi_\tau\| \leq 1$. Let $\Lambda_k = \lambda \mathbf{I} + \sum_{\tau=1}^k \phi_\tau \phi_\tau^\top$. Then for any $0 < \delta < 1$, with probability at least $1 - \delta$, we have for all $k > 1$,*

$$\left\| \sum_{\tau=1}^k \phi_\tau \eta_\tau \right\|_{\Lambda_k^{-1}}^2 \leq \sigma^2 d \log(1 + k/(d\lambda)) + \log \frac{1}{\delta}$$

When bounding the learning error for the transition model, we need to apply the above bound with respect to the value functions $v \in \mathcal{V}$, where we have the following lemma:

Lemma 7 (Lemma D.4 of [34]). *Let $\{x_\tau\}_{\tau=1}^\infty$ be a stochastic process on state space $\mathcal{S}$ with corresponding filtration $\{\mathcal{F}_\tau\}_{\tau=1}^\infty$. Let $\{\phi_\tau\}_{\tau=1}^\infty$ be a $\mathbb{R}^d$ -valued stochastic process with $\phi_\tau \in \mathcal{F}_{\tau-1}$, and $\|\phi_\tau\| \leq 1$. Let $\Lambda_k = \lambda \mathbf{I} + \sum_{\tau=1}^k \phi_\tau \phi_\tau^\top$. Then for any $\delta > 0$, and any $\tau > 0$ and $v \in \mathcal{V}$ that $\sup_x |V(x)| \leq H$, we have:*

$$\left\| \sum_{\tau=1}^k \phi_\tau \{V(x_\tau) - \mathbb{E}[V(x_\tau) | \mathcal{F}_{\tau-1}]\} \right\|_{\Lambda_k^{-1}}^2 \leq 4H^2 \left[\frac{d}{2} \log \left(\frac{k + \lambda}{\lambda} \right) + \log \mathcal{N}_\epsilon + \log \frac{1}{\delta} \right] + \frac{8k^2 \epsilon^2}{\lambda}.$$

where $\mathcal{N}_\epsilon$ is the ϵ -covering number of $\mathcal{V}$ with respect to the distance defined as the maximum distance over all states in $x \in \mathcal{S}$: $\text{dist}(V, V') = \sup_x |V(x) - V'(x)|$.

Given the linear MDP assumption, we have the following bound on the covering number of optimal value functions:

Lemma 8. *Denote the class of optimal value functions $V_{\mathcal{M}}^*$ as $\mathcal{V}$, where $\mathcal{M}$ is a linear MDP with fixed feature map ϕ defined in 1. Then the covering number of $\mathcal{V}$ can be bounded as:*

$$\log \mathcal{N}_\epsilon \leq d \log \left(1 + 4H\sqrt{d}/\epsilon \right)$$

Proof. Using Lemma B.1 of [34], we have for any policy π , the Q-function of a linear MDP with π can be written as $Q_h^\pi(s, a) = \langle \phi(s, a), w_h^\pi \rangle$, with $\|w_h^\pi\| \leq 2H\sqrt{d}$. And the value function can be written as: $V_h^*(s) = \max_a Q_h^*(s, a) = \max_a \langle \phi(s, a), w_h^* \rangle$. Applying a ϵ -covering argument over the Euclidean ball with radius $R = 2H\sqrt{d}$ for the vector set w_h^π , we have:

$$\begin{aligned} \text{dist}(V, V') &= \sup_s |V(s) - V'(s)| \\ &\leq \sup_{s, a} \langle \phi(s, a), w - w' \rangle \\ &\leq \|w - w'\| \\ &\leq \epsilon. \end{aligned}$$

Thus the ϵ -covering number of $\mathcal{V}$ can be bounded by the ϵ -covering number of the set of vectors w_h^π , which can be bounded as $\log \mathcal{N}_\epsilon \leq d \log \left(1 + 4H\sqrt{d}/\epsilon \right)$ following Lemma D.5 of [34]. $\square$

This lemma is used to bound the degree of the optimal function in the hard instance for imitation learning.

Lemma 9. *Consider a n -degree polynomial defined on $[\sin x, \cos x]$:*

$$f(x) = \sum_{u+v \leq n} g^{u,v} \sin^u x \cos^v x.$$

Then it has at most $2n$ roots in $x \in [0, 2\pi)$.

Proof. We can write $\sin x = \frac{z - z^{-1}}{2i}$ and $\cos x = \frac{z + z^{-1}}{2}$, where $z = \cos x + i \sin x$. Thus, $z^n f$ becomes a $2n$ -degree univariate polynomial in z . Since a univariate polynomial with degree $2n$ can have at most $2n$ complex roots, we have at most $2n$ roots for $x \in [0, 2\pi)$. $\square$

C Implementation Details

C.1 World Model

Here we provide the architecture details of our pose-image conditioned world model. Following [63], we add independent noise levels to the history frames with probability 0.5 and retain clear history frames with probability 0.5. For the future frames, we add uniform random noise and denoise them uniformly, without any auto-regressive schedule to speed up inference. We adopt multi-view prediction by concatenating 4 camera views into a grid, so that the model can infer the 3D positions of the robot. Specifically, for table top manipulators, we use wrist-view, front-view, side-view and top-view. For bi-manual robots, we use two wrist-views and the front-view and the side-view. For Robocasa, we utilize the 2 side-views, together with a wrist-view and top-view. Since the robot is mobile in Robocasa, we fix the top view camera, so we can infer the 2D movements from the top-view pose. We share the position embeddings of the frame tokens and pose image tokens, and only compute the flow matching loss of the frame tokens.

C.1.1 Pose-Image Conditioning: Decoupling Physical Simulation and Neural Imagination

While modern video foundation models generate high-fidelity and physically plausible videos [72], they often lack the dynamics necessary to predict the specific physical consequences of low-dimensional robot actions. These outcomes are inherently dependent on the robot’s unique kinematics and control parameters. To address this, we map abstract action vectors into visual joint pose skeletons, providing a semantically rich action representation. We compute anticipated joint positions by executing forward dynamics of the robot model. In practice, physical simulators such as MuJoCo, PyBullet, or Pinocchio can be employed to achieve these computations efficiently, given standard URDF or MJCF configurations. Although the robot configurations and physical dynamics are privileged information not widely used in previous world model implementations, we point out that this is essential for the world model to understand robot actions and generalize to novel actions.

We acknowledge that these forward-simulated poses are inaccurate, since external reaction forces introduced during object interaction are not captured by robot forward dynamics. For example, as illustrated in Fig. 6, a physical object may prevent a gripper from fully closing in the real environment, but the computed pose image will depict the gripper as closed because the commanded action was `close` and the reaction force from the object is unknown a priori. Also, when the gripper is holding objects or colliding with fixtures, the object mass and collision force can not be simulated via forward dynamics of the robot, resulting in inaccurate joint positions, especially for long action sequences. Crucially, these dynamics are not intended to be exact physical predictions. Instead, they serve as a mapping between actions and visual prompts such as pose-images. This pose-image based visual conditioning provides a generalizable, semantically grounded action representation that supplements the generative capabilities of video foundation models.

C.2 World Action Planner

To efficiently search for plausible actions within our world model, we adopt a global-local search strategy inspired by the framework in [83]. This approach employs a coarse-to-fine hierarchy: we utilize a larger scale for global action optimization guided by agent feedback, followed by a more fine-grained grid size for local search. The prompt templates for agent action proposal, feedback-driven optimization, and candidate ranking are provided in Figs 15, 16 and 17, respectively. During the global action optimization phase, we prompt the agent to provide directional modifications rather than precise coordinates, with the modification scale of 0.06. For `GridSearch`, we search the planar coordinates of the target gripper position across the set $\{(x, y), (x \pm \delta, y), (x, y \pm \delta)\}$, where the scale δ is set to 0.02 for `StackCube` and 0.05 for all other tasks. We set the *search scale (grid size)* to be approximately half the object size, so the candidate positions represent distinct manipulation poses around the object. When we need to modify the orientation, we search the target yaw angle within the set $\{\psi, \psi + 45^\circ, \psi - 45^\circ\}$. We emphasize that we search within the low-dimensional goal space rather than the space of individual actions, allowing for a significantly more compact search. The robot controller is implemented as a Diffusion Policy with a UNet backbone [10]. It accepts the current and target 7-dimensional end-effector poses as inputs to generate the action chunks required to reach the target end-effector state. For our experiments, we trained two policy variants with action chunk lengths of 40 and 20, respectively. We train the robot controller on noisy trajectories.

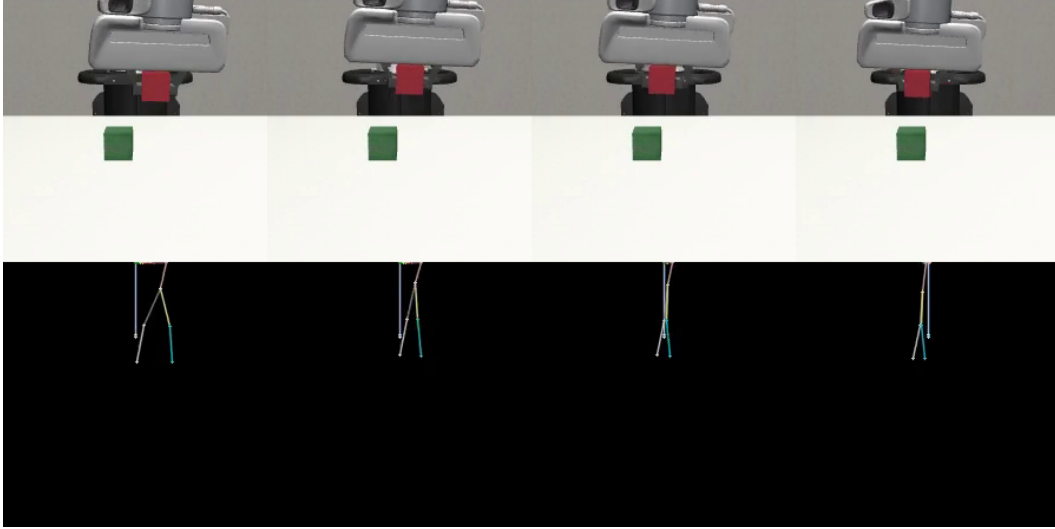


Figure 6: **Illustration of inaccurate pose image.** In the image, the first row is the actual environment rollout and the second row is the computed pose image using forward dynamics. Because the gripper action is `close` for every step, in the computed robot joint pose, the grippers jaws are closed immediately. In actual execution, due to the presence of the cube between the jaws, the opening between the jaws remain unchanged.

D Experiment Details

In this section, we provide detailed setup and more qualitative results for our experiments and ablations.

D.1 World Model

Here we provide the details of the action-conditioned world model experiments in Section 5.1.

D.1.1 Setup

To simulate the action sampling typical of the widely used Model Predictive Path Integral (MPPI) algorithm and to improve model generalization across novel exploratory actions, we augment expert demonstrations with Gaussian noise prior to trajectory replay. Specifically, we generate 100 trajectories per task: 40 from clean demonstrations, 40 with Gaussian noise added to the actions ($\sigma = 0.08$), and 20 with higher-intensity noise ($\sigma = 0.16$). We observe that model-based exploration primarily necessitates spatial reasoning regarding the end-effector’s location; thus, for the high-variance noise ($\sigma = 0.16$), we limit perturbations to the first three dimensions of the action vector (corresponding to target gripper positions) while maintaining fixed orientations. Incorporating such varied trajectories, including potential failures, is a standard practice in the literature [14, 29] for improving world model robustness. The model is trained with a global batch size of 64 on 16 H100 GPUs. For the cross-embodiment baselines, we adopt cross-attention as our base action-conditioning mechanism due to its superior performance (Table 1), and then apply the cross-embodiment-unification methods. Since our conditioning mechanism does not require any text input, we use a dummy text embedding for the Wan-T2V model for all inputs.

Sparse History. In our implementation, we utilize a sampling rate of 7 FPS for history frames, in contrast to the 20 FPS used for prediction frames and environment control. This sparse history allows for a longer temporal window without increasing the total number of history frames. Additionally, utilizing sparse frames helps mitigate spurious correlations rooted in local dynamics between continuous frames, forcing the model to learn the global environment con-

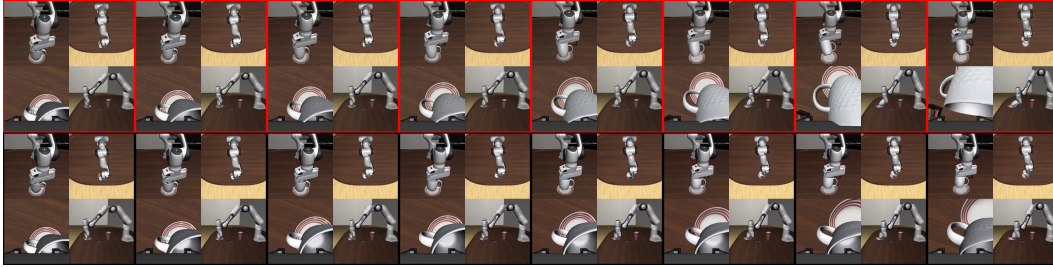
History FPS \ Dataset	LIBERO-Long	
	wrist-view	third-view
20 FPS (dense)	15.68 / 0.326	22.04 / 0.094
7 FPS (sparse)	15.98 / 0.320	22.14 / 0.093

Table 6: Ablation of sparse history frames

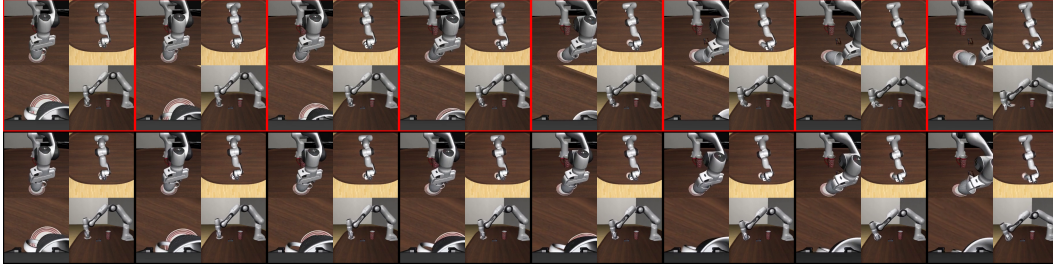
figuration rather than relying on short-term motion patterns. As shown in Table 6, this approach improves performance when generalizing to novel action sequences in LIBERO-Long.

D.1.2 Qualitative Results

We first evaluate the generalization capabilities of our pose-image conditioned world model by demonstrating its ability to predict the physical consequences of novel actions that lead to failures not present in the training set. As illustrated in Fig. 7, the world model successfully imagines the physical outcomes of actions that knock a mug over in various directions, such as backward or to the right. Notably, while Gaussian noise was incorporated during training, the noise levels were minimal and insufficient to produce such collisions; therefore, these "knocking over" trajectories are strictly out-of-distribution, highlighting our model’s capacity for zero-shot physical simulation and generalization. We further demonstrate the cross-embodiment modeling capabilities of our



(a) Qualitative results for knocking the mug over backwards.



(b) Qualitative results for knocking the mug over to the right.

Figure 7: Qualitative results for OOD actions that knocks the mug over, with the first row being ground truth frames with red borders, and the second row being generated imagination frames. The videos are downsampled 5x from 40 frames (2 seconds) to 8 frames.

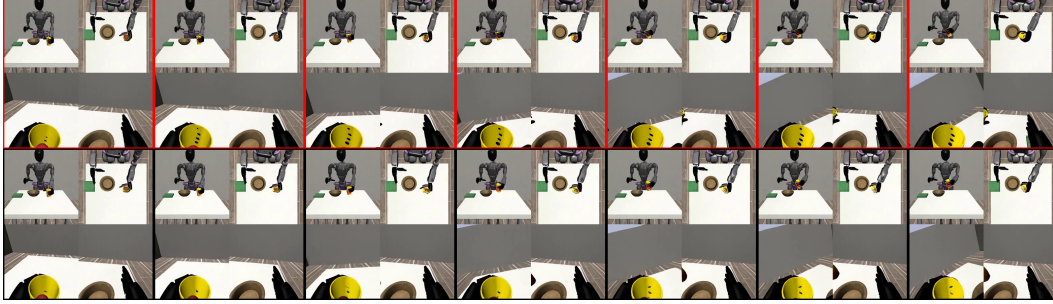
framework in Fig. 8. Our pose-image conditioned model successfully captures the dynamics across diverse robotic platforms, effectively generalizing from high-degree-of-freedom humanoid hands to mobile manipulators with side view by leveraging the embodiment-agnostic nature of the pose-image representation.

D.2 World Action Planner

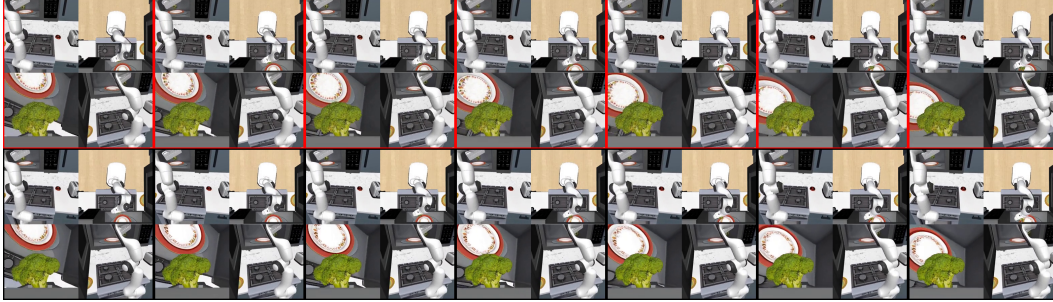
Here we provide the details of the world action planner experiments in Section 5.2.

D.2.1 Compositional Task Generalization

World Action Planner Setup. We train our diffusion policy, the $\pi_{0.5}$ baseline, and the cosmos-policy using an action chunk length of 40 (equivalent to a 2-second horizon) following [35], maintaining a 10-step execution horizon for each chunk before re-planning the subsequent sequence. Following [10], we implement our diffusion policy using a transformer architecture, with the specific hyperparameters detailed in Table 7. The policy is trained on the LIBERO-90 dataset using 30 expert demonstrations per task. Following [36], we condition the policy on task instructions via DistilBERT [61] embeddings. Because our model is trained from scratch with these low-dimensional embeddings, it lacks the inherent zero-shot generalization to novel instructions found in VLAs that utilize pre-trained VLM backbones. Consequently, our World Action Planner leverages the VLM agent to



(a) Qualitative results for dexterous humanoid in DexMimicGen dataset.



(b) Qualitative results for the Panda Omron robot in the Robocasa dataset.

Figure 8: Qualitative results cross-embodiment modeling, with the first row being ground truth frames with red borders, and the second row being generated imagination frames. The videos are downsampled 3x from 20 frames (1 second) to 7 frames.

Hyper-parameter	Value
Layers	12
Heads	8
Dimension	512
Observation length	1
Prediction length	40
Execution action steps	10
Input images	wrist view and agent view
Image resolution	128x128
Crop size	116x116
Batch size	64

Table 7: Details of our diffusion policy

decompose compositional task instructions into a sequence of atomic sub-tasks that the diffusion policy can execute. The world model is trained for 20k steps with a global batch size of 64, using 100 trajectories per task from the LIBERO-90 suite exclusively, as per the protocol in Section 5.1.

Baseline Details. The baselines follow the same configuration as our diffusion policy, utilizing an action chunk length of 40 and a 10-step execution horizon. These models are trained on the LIBERO-90 dataset, with 40 to 50 demonstrations per task after filtering out unsuccessful demonstrations and no-op actions following [36].

We train the $\pi_{0.5}$ baseline for 40k steps using the official LIBERO JAX implementation, achieving an average in-domain success rate of 95.8% on the LIBERO-90 tasks. Evaluation on the LIBERO-Long suite indicates that while $\pi_{0.5}$ achieves high success rates on non-compositional tasks which only involves environmental variations, it struggles with compositional generalization, particularly in scenarios requiring active navigation toward secondary targets. Detailed performance metrics are summarized in Table 8. From the results we can see that successes typically occurred when test tasks closely mirrored the training distribution. For instance, the task "STUDY SCENE1: pick up the book and place it in the back compartment of the caddy" in LIBERO-Long has a direct counterpart

Task	Success Rate
LIVING ROOM SCENE2 put both the alphabet soup and the tomato sauce in the basket	4
LIVING ROOM SCENE2 put both the cream cheese box and the butter in the basket	0
KITCHEN SCENE3 turn on the stove and put the moka pot on it	6
KITCHEN SCENE4 put the black bowl in the bottom drawer of the cabinet and close it	18
LIVING ROOM SCENE5 put the white mug on the left plate and put the yellow and white mug on the right plate	0
STUDY SCENE1 pick up the book and place it in the back compartment of the caddy	90
LIVING ROOM SCENE6 put the white mug on the plate and put the chocolate pudding to the right of the plate	0
LIVING ROOM SCENE1 put both the alphabet soup and the cream cheese box in the basket	0
KITCHEN SCENE8 put both moka pots on the stove	0
KITCHEN SCENE6 put the yellow and white mug in the microwave and close it	0

Table 8: Performance of $\pi_{0.5}$ trained on LIBERO-90 and evaluated on LIBERO-Long

in LIBERO-90 ("STUDY SCENE2"), with only minor environmental variations. In more complex scenarios, the gripper stalls after completing the first sub-task, while rarely drifting towards the second target object, resulting in near-zero success rates. These results reinforce our observation that imitation learning policies rely heavily on memorized motion patterns and lack the test-time reasoning necessary to synthesize novel trajectories for adaptation. For the cosmos-policy baseline, constrained by computational resources, we trained the model following the official setup for 30k steps, achieving an action L1 loss of 0.025. This configuration yields an average in-domain success rate of 93% on LIBERO-90 tasks. We found cosmos-policy has zero success rates on all compositional tasks in LIBERO-Long, except the "STUDY SCENE1: pick up the book and place it in the back compartment of the caddy" task which has a direct counterpart in LIBERO-90.

When evaluating policy enhancement methods such as SAILOR and GPC-RANK [29, 56]—which utilize policy-generated actions rather than our VLM agent proposals—we observe near-zero success rates. This occurs because the actions generated by the baseline policy have near zero magnitude in our compositional task generalization after finishing the first sub-task. While the Gaussian noise in SAILOR’s MPPI exploration [29] occasionally drifts the gripper toward the next object, it rarely results in successful task completion. These results highlight the importance of agent intervention in our system.

D.2.2 New Layout Generalization

Layout Setup. We provide the images of the original layout and our modified layout below in Fig. 9.

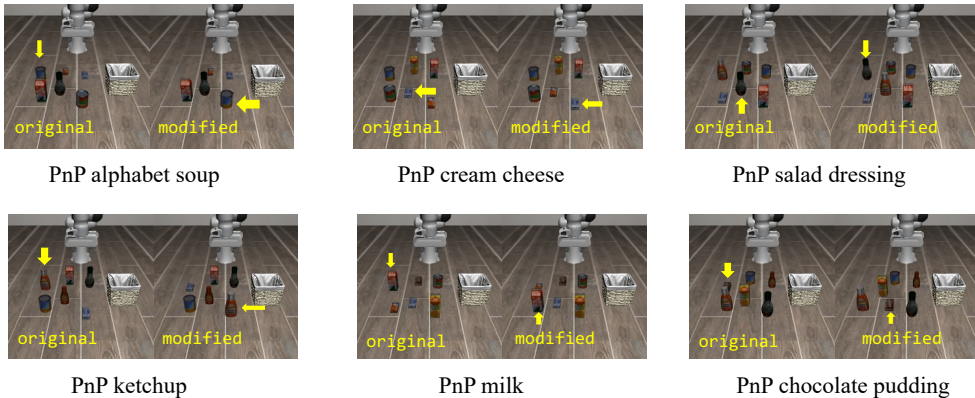


Figure 9: Original object layout and our modified object layout in LIBERO-Object. Yellow arrow points to the target object to be grasped into the basket.

World Action Planner Setup. For our method, we fine-tune the diffusion policy and world model (pre-trained on LIBERO-90 as in the previous compositional task generalization experiments) using the official LIBERO-Object dataset under its original layout. The diffusion policy is fine-tuned with only 5 demonstrations per task. To enhance the robustness of the world model, we augment the fine-tuning set with 10 additional trajectories generated by adding Gaussian noise (with std of 0.08 and 0.16) to the expert actions.

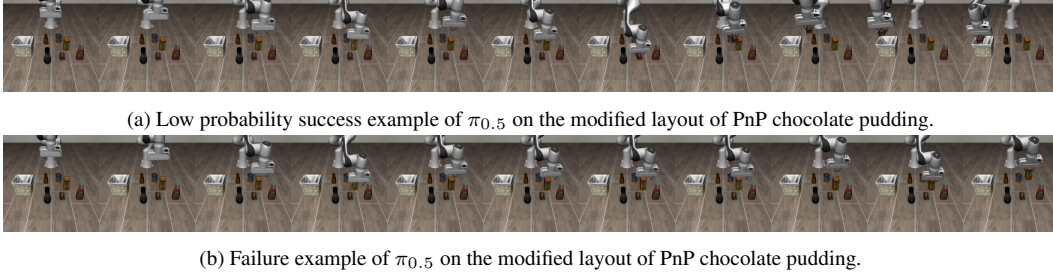


Figure 10: Qualitative evaluation of $\pi_{0.5}$ on the modified layout of PnP chocolate pudding, down-sampled from 20 FPS to 1 FPS for clarity. The VLA exhibits low success rates, as the gripper rarely drifts toward the correct target and instead frequently grasps distractor objects placed at the original position during re-grasp attempts.

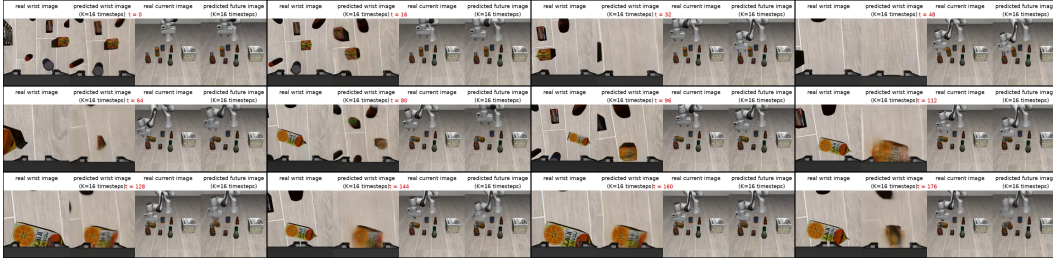


Figure 11: Qualitative evaluation of the cosmos-policy on the modified layout of the PnP chocolate pudding task, illustrating ground-truth environment rollouts alongside predicted future states from the WAM. Despite the modified layout, the model continues to target the original target object coordinates, resulting in repetitive and unsuccessful re-grasp cycles at the original position of the target object.

Baseline Setup. We directly evaluate the official LIBERO checkpoints for the $\pi_{0.5}$ and cosmos-policy baselines. We observe that $\pi_{0.5}$ achieves non-zero success rates on the “PnP Chocolate Pudding” task because the target object is not sufficiently displaced from its original training location; this allows the policy to occasionally capture the pudding during subsequent re-grasp attempts, as shown in Fig. 10a. In most other cases, however, the policy fails by erroneously targeting distractor objects near the original coordinates, as illustrated in Fig 10b. For other tasks where the target object is displaced far away, the $\pi_{0.5}$ model exhibits zero success rates, with similar patterns of grasping distractor objects placed near the original coordinates.

The cosmos-policy baseline performs worse than $\pi_{0.5}$, yielding a zero success rate across all tasks. Notably, the WAM consistently repeats grasp attempts at the original training coordinates, failing to exhibit the local exploration observed in $\pi_{0.5}$. Qualitative results of the environment rollout and the predicted future states from the WAM are illustrated in Fig. 11. While the predicted images are visually realistic, they do not facilitate successful task completion. These results suggest that because the model is trained primarily to mimic demonstration trajectories and predict action outcomes based on pre-trained video generation, it lacks the necessary reasoning and planning capabilities to generalize to novel tasks or environmental layouts.

For policy enhancement methods such as SAILOR and GPC-RANK [29, 56], we also observe near-zero success rates because the actions generated by the policy often target incorrect positions. These results underscore the importance of independent reasoning and planning when encountering novel scenarios, rather than relying exclusively on the demonstrations within the imitation learning dataset.

D.2.3 Zero-shot generalization

In the zero-shot setting, we hard code the GRASP and RELEASE action sequence as below, where the actions are represented as end-effector operational space velocity control:

- GRASP: $[0, 0, -0.5, 0, 0, 0, -1] \times 10 + [0, 0, 0, 0, 0, 0, 1] \times 10 + [0, 0, 1, 0, 0, 0, 1] \times 10$
- RELEASE: $[0, 0, 0, 0, 0, 0, -1] \times 10 + [0, 0, 1, 0, 0, 0, -1] \times 10$

To fine-tune our world model, we utilize the LIBERO-90 model as a backbone and collect 50 exploratory trajectories based on actions proposed by the VLM. The world model is then fine-tuned for 10 epochs. We point out that although successful trajectories exist among the exploratory trajectories, we only use them to finetune the world model, while prior methods [29, 56] require additional expert demonstrations beside the in-distribution sampled trajectories for success.

D.2.4 Ablation Experiments

Here we provide the results for the ablation experiments in Section 5.2.4.

Ablation of individual components. We first conduct an ablation study to evaluate the individual contributions of each module within the World Action Planner. The results, presented in Table 9, demonstrate that each component is essential for maximizing overall performance. For easy tasks, object clearance via global action optimization is sufficient. Notably, for the PnP alphabet soup and

Method \ Task	PnP alphabet soup & tomato sauce	PnP white mug & yellow and white mug	PnP white mug & chocolate pudding	PnP alphabet soup & cream cheese box
Vision-language planner	56	28	46	32
+ Global action optimization	64	40	70	48
+ Local action search	64	54	72	66
+ Policy rollout imagination	72	68	78	70

Table 9: **Results for ablations of our action refinement pipeline.** We ablate different components in our world action planner using the LIBERO-Long tasks, where each component contributes to higher success rates.

cream cheese task, when grasping the cream cheese box after placing the alphabet soup, the gripper is rotated, requiring the agent to find the right orientation for the grasp. While the VLM struggles to describe the required rotation analytically, it can effectively identify the optimal orientation when evaluating imaginations of candidate poses, as illustrated in Fig. 12. For the fine-grained PnP white

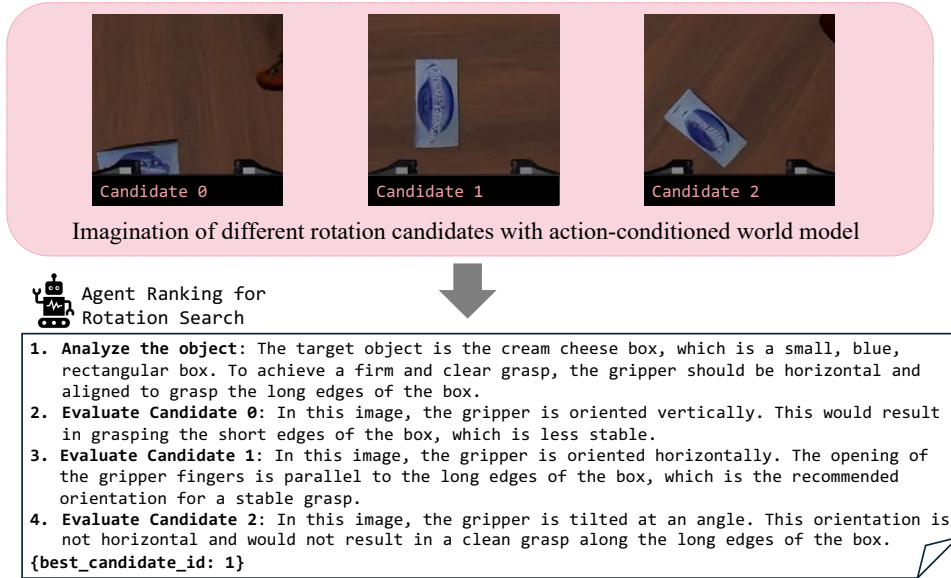


Figure 12: Illustration of local search for gripper rotation. The VLM successfully identifies the right rotation from imagination of candidates, while failing to describe the correct rotation analytically.

mug and yellow-and-white mug task, transitioning from the first mug to the next presents a specific challenge: it is often unclear which state serves as a valid "in-distribution" starting point for the policy to successfully trigger a grasp and complete the second sub-task. To resolve this ambiguity, we imagine consecutive policy rollouts following various action candidates to determine which state allows the policy to effectively continue the task. As illustrated in Fig. 13, we can identify the optimal state for completing the grasp by imagining the consecutive policy rollout (shown in row 2), whereas selecting the correct candidate remains difficult without the benefit of further imagination (as in row 1).

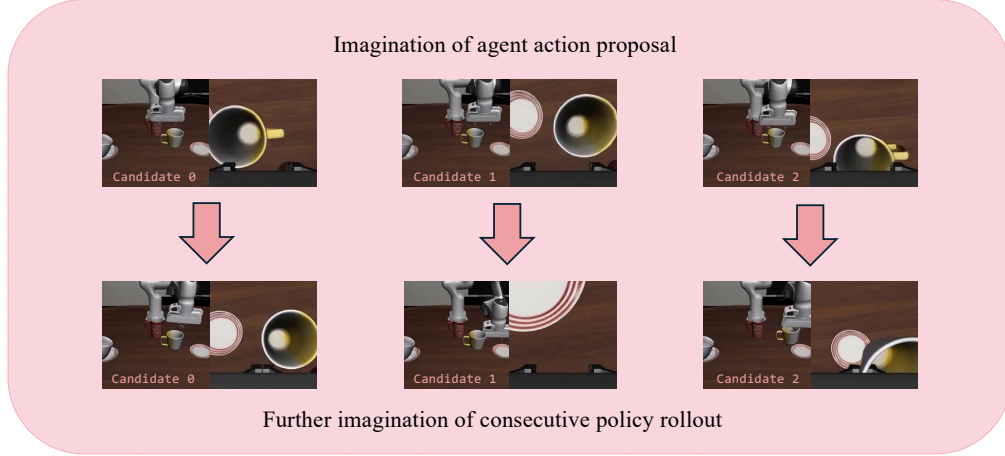


Figure 13: Illustration imagining consecutive policy rollout. While in the first row, it is hard to identify which one is the good state to continue the second sub-task by grasping the mug, we can easily see that for Candidate 2, the policy successfully grasped the mug and continues the second sub-task from the consecutive policy rollout imagination in the second row.

Ablation against Best-of-N Sampling. Next, we ablate the efficiency of our optimization and search pipeline against naive best-of-N sampling from the VLM. To provide a rigorous upper bound, for best-of-N sampling we use ground-truth rewards by executing the actions in the environment and observe the environment reward, while we restrict to world model imaginations and VLM agent evaluations for our method. The results are shown in Table 10. We observe that VLMs frequently

Method	BoN-1	BoN-2	BoN-4	BoN-8	Global optimization
Success %	16	24	36	42	60
Imaginations #	0	2	4	8	1

Table 10: **BoN Sampling vs. Our Planning Pipeline.** BoN- i denotes best-of-N with i samples. In the PnP ketchup task, the VLM fails to account for physical constraints across multiple samples, resulting in collision and unsafe execution; while our global action optimization effectively identifies such risks and corrects the trajectory for a safe and successful completion.

exhibit systematic physical oversights; e.g., in the LIBERO-Object "PnP ketchup" task, the VLM often fails to lift the object enough to clear the basket rim, a failure that persists across multiple samples. In contrast, as illustrated in Fig. 14, our approach identifies these physical violations and rectifies the trajectory through global optimization with agent feedback, requiring only much fewer imaginations.

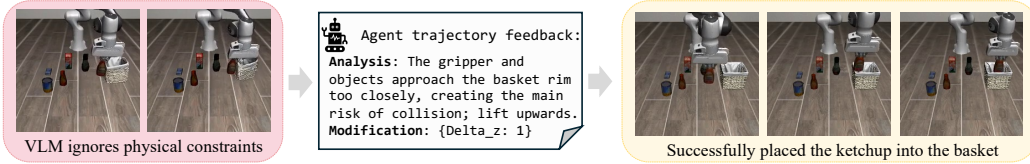


Figure 14: **Global Action Optimization.** While VLMs often ignore physical constraints, our framework leverages world model imaginations to identify potential collisions and optimize action sequences for safe execution.

We also compare our local search with GridSearch exclusively against the BoN baseline in the StackCube task, where global optimization for physical clearance is not essential and pure local search alone is sufficient to complete the task successfully, although occurring larger computation

cost with more imaginations. As shown in Table 11, even with similar sample-and-select logic, our GridSearch outperforms BoN sampling by actively exploring the neighborhood of the actions, while VLMs can make repetitive errors across multiple samples in certain states.

Method	BoN-1	BoN-2	BoN-4	BoN-8	BoN-10	Local search
Success %	22	28	32	50	62	70
Imaginations #	0	2	4	8	10	6

Table 11: **BoN Sampling vs. Our Local Grid Search.** BoN- i denotes best-of- N with i samples. In the StackCube task, our GridSearch outperforms BoN sampling via actively searching the neighborhood of the actions, while VLMs can make repetitive errors in certain states.

E Wall Clock Time Analysis

Here we report the wall clock time of our world model. When training on a H100 GPU with batch size 2, one single training step takes approximately 4.8 seconds. When running inference on a A100 GPU with batch size 1, one single forward takes 0.85 seconds, so predicting 20 frames requires 20 denoising steps and takes around 17 seconds. As a result, the global optimization will take around 30 seconds including world model imagination and VLM reasoning, and the local search can take 2 to 3 minutes. We note that the computation cost of local search can be significantly decreased by adopting an auto-regressive world model and using KV cache for the shared history frames among all the candidates, which we leave as future work.

We emphasize that while generating imagined rollouts with the world model is computationally intensive, the system does not require continuous re-planning at every control step. In our experiments, we only invoke the planning system at critical decision points: once during the transition between sub-tasks in compositional settings, and twice, specifically at the onset of the grasping and placement phases, for other evaluation tasks. Furthermore, the overall planning latency is significantly influenced by the VLM agent’s reasoning time. This process is analogous to “System 2” reasoning in LLM agents, where a deliberate thinking phase and chain-of-thought reasoning is required to solve difficult problems [28, 41].

You are a helpful assistant for controlling a robot arm in a simulated environment. You will give advice on how to complete the task based on the image inputs.

Task instruction. Here is the task instruction: {detailed_task_instruction}

Observation. Here is the starting state of the current episode <images> and here is the current state images <images>. The execution history so far is <images>.

Action Proposal. Now, please propose the next actions for the robot to complete the task. You should complete the task following the order in the instruction. The available atomic actions are:

- **MOVE** Here you move the gripper to a target position, and you should point it out in the multiview current state images. The position should be represented by x,y pixel coordinates normalized to 0-1000. To grasp or operate an object, move towards it. If you need to place an object, point to the target gripper position so the object can be placed following the instruction. Examples:

```
{"action": "MOVE",  
"parameters": {"frontview": {"x": 500, "y": 300},  
"topview": {"x": 450, "y": 350},  
"sideview": {"x": 480, "y": 320}}}
```

- **ROTATION** Here you rotate the gripper, and you return the rotation in Euler angles [delta_roll, delta_pitch, delta_yaw] in degrees. The coordinate system and axis are defined as coordinate_system_description. Example:

```
{"action": "ROTATE", "parameters": {"delta_roll": 0,  
"delta_pitch": 15, "delta_yaw": 0}}
```

- **RELEASE** Here you release the object by opening the gripper.

Example: {"action": "RELEASE", "parameters": {}}

- **GRASP** Here you grasp the object by closing the gripper.

Example: {"action": "GRASP", "parameters": {}}

Finally, return a list of actions in json format.

Figure 15: Example prompt template for agent action proposal.

Optimize gripper position. Here is the gripper position of our next robot actions <images>, and I want you to look carefully and analyze the position of the gripper and the object, and optimize the gripper position following the guidelines below. Return the gripper position adjustments in x y z directions: The coordinate system and axis are defined as {coordinate_system_description}.

Guidelines.

- Understand what the gripper is trying to do, based on your task instruction understandings. {detailed_task_description}.
- Analyze the trajectory frame by frame, and inspect whether there may be potential collisions with any object along the trajectory. Make sure that the gripper had cleared any previous objects such as cups and bucket rims during the movement. If there is any potential collision with rims and objects, adjust the gripper so that the trajectory is fully clear of any obstacles.
- When the gripper is trying to place the object, first verify whether the position is above the target region to be placed. Zoom in on the frontview image, check whether the gripper is clearly to the left or right of the correct target region, and return the adjustment in y direction. Zoom in on the sideview image, check whether the gripper is clearly to the left (close to the camera) or to the right (away from the camera), and return the adjustment in x direction.
- If the gripper is about to grasp an object, the gripper should be aligned and directly above the object. Adjust the gripper if it is clearly misaligned with the object, with both gripper jaws outside the object.

Zoom in on the frontview image, and see if the gripper is below or above the object. The gripper should be slightly above the object top to ensure enough room for descending and grasping. If the gripper jaws are almost touching the object in the final images and there is potential risk of collision, you should lift the gripper by returning 1 in the z direction. Check whether the gripper is to the left or right of the object with both jaws outside the object, and return the adjustment in y direction.

Zoom in on the wristview image to see if the object is between the jaws of the gripper. If not, move the gripper so that it is above the object and well aligned with the object. {wrist_view_coordinate_system_description}

Zoom in on the side-view image to see whether the gripper jaws are centered and above the object, adjust the position if the gripper jaws are clearly outside the object. {sideview_coordinate_system_description}

Format. Please return your optimization direction in json format, for example:

```
{"Delta_x": 0, "Delta_y": -1, "Delta_z": 1}
```

where x, y, z can only be -1, 0 or 1, with 0 being no movement in that direction, 1 being move towards positive direction and -1 being move towards negative direction. Return 0 in the corresponding axis if the gripper is mostly aligned and safe.

Figure 16: Example prompt template for agent feedback for action optimization.

I have a set of images <images> showing the robot actions trying to complete the task {detailed_task_description}. I want you to rank them from best to worst in terms of whether the action is accomplishing the task goal safely. Here are more detailed guidelines:

Guidelines.

- If the gripper is trying to grasp an object, I want you to rank them from best to worst in terms of whether the grasp is firm and clear. The best image should be a clear and firm grasp at the right place.
Reflect on the trajectory history and task instructions, and understand which object is the gripper trying to grasp.
Identify the position of the jaws of the gripper in the wristview image, which is at the bottom. Verify whether the {grasping_position_of_target_object} is being grasped between the jaws clearly.
- If the gripper orientation is different in the candidate images, you should identify the best orientation for a clear and stable grasp.
- If the gripper is trying to place {grasp_object} onto / into {target_object}, the {grasp_object} and gripper should be directly above the center of the {target_object}, so in the front-view and side-view images, the gripper and object should be above and within the {target_object}. If the object and gripper is to the left or to the right outside of the {target_object}, the grasped object may be dropped out of the {target_object}, so the position is bad.

Reminder.

- You should rank all the images following the same standard. If none of them is perfect, you should rank them by which one is closest.
- When ranking the later images, refer and reflect the previous candidates to rank them faithfully. For example, if the first image is blurry and the second image is clear, then the second image should be ranked higher than the first image.

Return the ranking result in json format, for example: [0, 2, 1], where the ids in the list are the image ids ranked from best to worst, with the first being the best. The range of the ids should be from 0 to N-1, where N is the total number of candidate images.

Figure 17: Example prompt template for agent candidate ranking for local search.